

Mecanum omnidirectional robot: kinematics, odometry and experimental validation in simulation

Course: Mobile Robotics

Guerrero Schmidt, Mateo

Muñoz, Joaquín Eliseo

Antonio, Cristian Ignacio

Contents

1	Introduction	4
2	Kinematic model	4
2.1	Features of the platform	4
2.2	Parameters and conventions	5
2.3	Inverse kinematics	6
2.4	Forward kinematics	7
3	Odometry	7
3.1	Encoder reading	8
3.2	Transformations between reference frames	8
3.3	Pose integration	9
4	Closed-loop control and trajectory tracking	10
4.1	Error computation	10
4.2	Proportional control	11
4.3	Trajectory tracking	12
5	EKF-based localization	13
5.1	State and motion model	13
5.2	Landmark detection	14

5.3	Observation model	15
5.4	Association of measurements with landmarks	16
5.5	EKF correction	16
6	Experimental results	19
6.1	Validation of the kinematic model	19
6.1.1	Sequence 1: velocity along the x axis	20
6.1.2	Sequence 2: velocity along the y axis	21
6.1.3	Sequence 3: angular velocity	22
6.1.4	Sequence 4: combined linear velocities	23
6.1.5	Comparison between odometry and ground truth	24
6.2	Controller tuning: comparison of configurations	26
6.2.1	Detailed analysis of the selected configuration	32
6.3	Manual tests of EKF-based localization	35
6.3.1	Circular motion	36
6.3.2	Manual square with longitudinal and lateral motions	39
6.3.3	In-place rotation	42
6.3.4	Overall comparison	44
6.4	Closed-loop tracking with EKF feedback	46
6.4.1	Resulting trajectory	46
6.4.2	Time evolution of the pose	48
6.4.3	Path error	49
6.4.4	Velocities sent to the robot	51
6.4.5	Evolution of the covariance	51
6.4.6	Comparison with odometry-based tracking	55
7	Developed system	58
8	Conclusions	60
A	Velocities of the control experiments	61
B	Actual pose and control targets	63

1 Introduction

Omnidirectional robots have an important advantage over other mobile platforms: they can move across the plane in different directions without first having to reorient themselves. On a holonomic platform, such as one with Mecanum wheels, the robot can move forward, move sideways and rotate at the same time. This allows more direct motions in confined spaces, where a non-holonomic robot, for example a differential-drive one, usually needs rotate-translate-rotate maneuvers to reach the same pose.

This capability is useful in applications where the available space is limited or precise positioning is required, such as load transport, alignment with workstations or motion near obstacles. The robot can also move while keeping a given orientation, for example in inspection or mapping tasks.

In this work, a ROS2 system was implemented for an omnidirectional robot with Mecanum wheels, simulated in CoppeliaSim. The development includes the inverse and forward kinematics of the platform, odometry estimation from encoders, a closed-loop controller for trajectory tracking and an EKF-based localization stage that combines the odometric prediction with landmark observations. Finally, the system is validated through experiments that compare the target velocities with the actual response and evaluate the odometry and the EKF estimate against the simulator's ground truth.

The starting point for this implementation was the work developed during the semester to control a differential-drive robot. The control logic, the odometry estimation and the kinematic model had to be adapted to the case of an omnidirectional platform. In a differential-drive robot, lateral motion does not exist as an independent degree of freedom; in a Mecanum robot, it is part of the controllable motion.

2 Kinematic model

2.1 Features of the platform

The robot used has four Mecanum wheels. This type of wheel differs from a conventional wheel because it has inclined rollers, usually at 45° . As a result, when a wheel turns it generates both a longitudinal and a lateral component of motion. By combining the effect of the four wheels, the robot can produce three independent motions in the plane: forward or backward motion, lateral displacement and rotation.

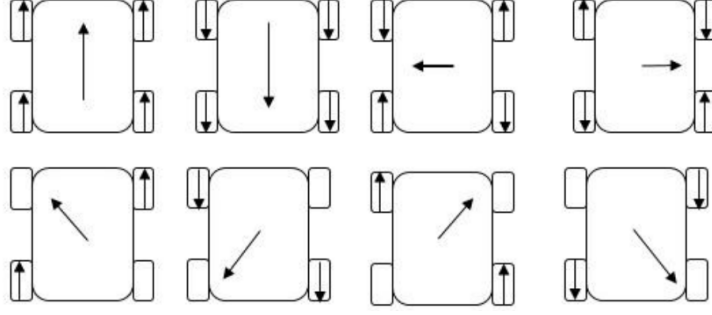


Figure 1: Motions of an omnidirectional platform. Adapted from Taheri et al. [1].

As shown in Figure 1, each motion is obtained from a different combination of the rotation directions of the four wheels.

In this work, the left superscript r indicates that a quantity is expressed in the robot frame, while w indicates that it is expressed in a global frame. In the odometry sections, this global frame corresponds to `odom`, while in EKF-based localization it may correspond to `map`. To simplify notation, θ is used instead of $^w\theta$ in the arguments of trigonometric functions and rotation matrices.

The chassis velocity, expressed in the robot frame, is represented by the vector

$${}^r\mathbf{v} = \begin{bmatrix} {}^rv_x \\ {}^rv_y \\ {}^r\omega_z \end{bmatrix},$$

where rv_x is the longitudinal velocity of the robot, rv_y the lateral velocity and ${}^r\omega_z$ the angular velocity about the vertical axis.

Unlike a differential-drive robot, where the wheel velocities determine only a linear velocity and an angular velocity, on a Mecanum platform the four wheels make it possible to control the three degrees of freedom of planar motion. A kinematic model is therefore needed to relate the chassis velocities to the angular velocities of each wheel. This model is used in both directions: to generate the target wheel velocities and to reconstruct the robot's motion from the encoders.

2.2 Parameters and conventions

To formulate the model, the following geometric and sensing parameters were defined:

- r : wheel radius [m].
- l_x : distance between the robot center and the wheels in the longitudinal direction [m].
- l_y : distance between the robot center and the wheels in the lateral direction [m].
- $a = l_x + l_y$: effective distance used in the rotation term.

- N : encoder resolution, measured in ticks per revolution.
- Δt : time between two consecutive encoder readings.

The simulation used a $0.35\text{ m} \times 0.35\text{ m}$ base, so $l_x = l_y = 0.175\text{ m}$. The wheel radius was taken as $r \approx 0.05\text{ m}$ and the encoder resolution as $N = 500\text{ ticks/rev}$. These values directly affect the conversion between ticks, wheel angular velocity and chassis velocity.

The wheel order used was:

- FL : front left wheel;
- FR : front right wheel;
- RL : rear left wheel;
- RR : rear right wheel.

Two reference frames are distinguished for the odometry. `base_link` is the local frame of the robot and `odom` is the global or inertial frame in which the odometry is integrated. The velocities obtained through forward kinematics are expressed in the robot frame, but the pose is updated in the global frame.

The forward and inverse kinematic relations were based on the model presented by Taheri et al. [1], adapted to the wheel order and notation used in this work.

2.3 Inverse kinematics

Inverse kinematics determines the angular velocity of each wheel from the desired motion of the chassis. That is, it transforms the components

$$({}^r v_x, {}^r v_y, {}^r \omega_z)$$

into the angular velocities of the four wheels

$$(\omega_{FL}, \omega_{FR}, \omega_{RL}, \omega_{RR}).$$

The relation is expressed as

$$\begin{bmatrix} \omega_{FL} \\ \omega_{FR} \\ \omega_{RL} \\ \omega_{RR} \end{bmatrix} = \frac{1}{r} \begin{bmatrix} 1 & -1 & -a \\ 1 & 1 & a \\ 1 & 1 & -a \\ 1 & -1 & a \end{bmatrix} \begin{bmatrix} {}^r v_x \\ {}^r v_y \\ {}^r \omega_z \end{bmatrix}. \quad (1)$$

Equation 1 shows that each wheel combines the three motion components differently. For example, to move sideways, some wheels must turn in one direction and others in the opposite direction; to rotate, something similar happens, but with a different combination.

2.4 Forward kinematics

Forward kinematics performs the opposite process: the chassis velocity is estimated from the angular velocities of the wheels. This is needed for odometry, because the encoders do not directly measure ${}^r v_x$, ${}^r v_y$ or ${}^r \omega_z$, but the motion of each wheel.

From these measurements, the components of the chassis velocity expressed in the robot frame are computed as

$${}^r v_x = \frac{r}{4} (\omega_{FL} + \omega_{FR} + \omega_{RL} + \omega_{RR}), \quad (2)$$

$${}^r v_y = \frac{r}{4} (-\omega_{FL} + \omega_{FR} + \omega_{RL} - \omega_{RR}), \quad (3)$$

$${}^r \omega_z = \frac{r}{4a} (-\omega_{FL} + \omega_{FR} - \omega_{RL} + \omega_{RR}). \quad (4)$$

The linear components can also be used to compute the angle γ_v , which indicates the direction of motion, and the magnitude v_R of the resulting velocity in the plane:

$$\gamma_v = \text{atan2}({}^r v_y, {}^r v_x),$$

$$v_R = \sqrt{({}^r v_x)^2 + ({}^r v_y)^2}.$$

These quantities were not used directly in the control law, since the controller works separately with the components ${}^r v_x$ and ${}^r v_y$.

The quality of the odometric estimate depends on two elements: that the kinematic model correctly represents the geometry of the robot and that the encoder measurements are consistent.

3 Odometry

The odometric estimate was built from the encoder measurements. At each instant, the tick increments of each wheel were computed and converted into angular velocities. Then, through forward kinematics, the robot velocity was estimated in its local frame. Finally, these velocities were transformed to the global frame and integrated over time to update the pose.

The goal of this stage is to estimate the pose

$${}^w \mathbf{x} = \begin{bmatrix} {}^w x \\ {}^w y \\ {}^w \theta \end{bmatrix},$$

where ${}^w x$ and ${}^w y$ represent the position of the robot in the global frame, while ${}^w \theta$ represents its orientation with respect to that frame.

3.1 Encoder reading

For each wheel $i \in \{FL, FR, RL, RR\}$, the tick increment between two consecutive instants is computed as

$$\Delta ticks_i(k) = ticks_i(k) - ticks_i(k-1).$$

The angular velocity of the wheel is computed from this increment. If the encoder has N ticks per revolution, then

$$\omega_i(k) = \frac{2\pi}{N} \frac{\Delta ticks_i(k)}{\Delta t}. \quad (5)$$

With Equation 5, the tick increment is converted into an angular velocity expressed in radians per second. Once the velocities of the four wheels are obtained, the forward kinematics defined in Equations 2, 3 and 4 is applied.

3.2 Transformations between reference frames

The velocities ${}^r v_x$ and ${}^r v_y$ obtained through forward kinematics are expressed in the local frame of the robot. However, to update the position (${}^w x, {}^w y$) they must be expressed in the global frame, using the current orientation of the robot ${}^w \theta$.

The velocity vector is transformed from the robot frame, denoted by r , to the global frame, denoted by w , using the rotation matrix

$$\underbrace{\begin{bmatrix} {}^w \dot{x} \\ {}^w \dot{y} \end{bmatrix}}_{{}^w \mathbf{v}} = \underbrace{\begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}}_{{}^w \mathbf{R}_r(\theta)} \underbrace{\begin{bmatrix} {}^r v_x \\ {}^r v_y \end{bmatrix}}_{{}^r \mathbf{v}}. \quad (6)$$

Expanding the transformation defined in Equation 6 gives

$$\begin{aligned} {}^w \dot{x} &= {}^r v_x \cos(\theta) - {}^r v_y \sin(\theta), \\ {}^w \dot{y} &= {}^r v_x \sin(\theta) + {}^r v_y \cos(\theta), \\ {}^w \dot{\theta} &= {}^r \omega_z. \end{aligned}$$

The angular component does not require this rotation, since ${}^r \omega_z$ represents the rotation rate about the vertical axis, which is common to both frames in planar motion.

No translation term appears in the previous transformation because what changes is the frame in which a velocity vector is expressed, not the position of a point. To transform the position of a point from the robot frame to the global frame, the global position of the robot origin would also have to be added

$${}^w\mathbf{p} = \underbrace{\begin{bmatrix} {}^wx \\ {}^wy \end{bmatrix}}_{{}^w\mathbf{t}_r} + {}^w\mathbf{R}_r(\theta) {}^r\mathbf{p}.$$

In this expression, ${}^r\mathbf{p}$ and ${}^w\mathbf{p}$ represent the position of the same point expressed in the robot frame and in the global frame, respectively.

The inverse transformation is used to express the velocity vector in the robot frame. Since ${}^w\mathbf{R}_r(\theta)$ is orthogonal, its inverse is equal to its transpose

$${}^r\mathbf{v} = {}^r\mathbf{R}_w(\theta) {}^w\mathbf{v} = ({}^w\mathbf{R}_r(\theta))^T {}^w\mathbf{v} = \begin{bmatrix} \cos(\theta) & \sin(\theta) \\ -\sin(\theta) & \cos(\theta) \end{bmatrix} \begin{bmatrix} {}^w\dot{x} \\ {}^w\dot{y} \end{bmatrix}.$$

For the position of a point, going from the global frame to the robot frame requires first subtracting the position of the robot origin and then applying the inverse rotation

$${}^r\mathbf{p} = {}^r\mathbf{R}_w(\theta) ({}^w\mathbf{p} - {}^w\mathbf{t}_r) = ({}^w\mathbf{R}_r(\theta))^T \left({}^w\mathbf{p} - \begin{bmatrix} {}^wx \\ {}^wy \end{bmatrix} \right).$$

In the previous expressions, w generically represents the reference frame in which the robot pose is expressed. The implementation used the `map` and `odom` frames. The first is associated with the map of the scene and the known positions of the posts; the second is used as the reference for integrating the odometry from the start of each run. Both are related by a fixed transformation published by TF, with a constant translation and rotation. This transformation makes it possible to express a pose in either of the two frames.

To compute the error between the desired pose and the estimated pose, both must first be expressed in the same reference frame. It would not be correct to directly subtract a pose expressed in `map` from one expressed in `odom`. In the odometry experiments, both poses were expressed in `odom`. When the EKF estimate (published in `map`) was used, it was first transformed to `odom` to carry out the comparison.

The inverse rotation is used later to express in the robot frame the position error computed in the global frame. In that case no translation is added either, because the error is the difference between two positions and behaves as a vector.

3.3 Pose integration

Once the velocities in the global frame are obtained, they are integrated over time using a discrete Euler approximation

$${}^w x_{k+1} = {}^w x_k + {}^w \dot{x}_k \Delta t, \quad (7)$$

$${}^w y_{k+1} = {}^w y_k + {}^w \dot{y}_k \Delta t, \quad (8)$$

$${}^w \theta_{k+1} = {}^w \theta_k + {}^r \omega_{z,k} \Delta t. \quad (9)$$

Equations 7, 8 and 9 give an incremental estimate of the robot pose. This estimate is published as odometry and can then be used by the controller or by other modules of the system. Odometry accumulates errors over time: any small error in the wheel measurements, in the kinematic model or in the integration propagates as the trajectory progresses.

4 Closed-loop control and trajectory tracking

The kinematic model makes it possible to convert the desired chassis velocities into wheel velocities and to estimate the robot motion from the encoders, but it does not determine which velocities must be applied to reach a pose or follow a trajectory. For this reason, a closed-loop controller was implemented that, at each instant, compares the desired pose with the estimated pose and updates the commands to reduce the error.

Since the Mecanum platform can move longitudinally and laterally, in addition to rotating, the controller acts directly on the three degrees of freedom of the plane: v_x , v_y and ω_z . Unlike a differential-drive robot, the chassis does not need to be oriented first in order to correct a lateral displacement.

4.1 Error computation

Let the current pose of the robot be

$${}^w \mathbf{x} = \begin{bmatrix} {}^w x \\ {}^w y \\ {}^w \theta \end{bmatrix},$$

and let the desired pose be

$${}^w \mathbf{x}_d = \begin{bmatrix} {}^w x_d \\ {}^w y_d \\ {}^w \theta_d \end{bmatrix}.$$

The position error in the global frame is computed as

$${}^w \mathbf{e}_p = \begin{bmatrix} {}^w x_d - {}^w x \\ {}^w y_d - {}^w y \end{bmatrix}.$$

To control the motion of the robot, it is convenient to express this error in the local frame of the chassis, so that the longitudinal component is associated with ${}^r v_x$ and the lateral one with ${}^r v_y$. The inverse rotation is then applied

$${}^r \mathbf{e}_p = \underbrace{\begin{bmatrix} \cos(\theta) & \sin(\theta) \\ -\sin(\theta) & \cos(\theta) \end{bmatrix}}_{{}^r \mathbf{R}_w(\theta)} \begin{bmatrix} {}^w x_d - {}^w x \\ {}^w y_d - {}^w y \end{bmatrix}. \quad (10)$$

The angular error is defined as

$$e_\theta = {}^w \theta_d - {}^w \theta.$$

In the implementation, this error was normalized to avoid artificial jumps when the angle crosses the boundary between π and $-\pi$.

4.2 Proportional control

Based on the error, a proportional controller was used: the larger the difference between the current pose and the desired pose, the higher the velocity sent to the robot. The control law used is given by Equation 11

$${}^r \mathbf{u} = \begin{bmatrix} {}^r v_x \\ {}^r v_y \\ {}^r \omega_z \end{bmatrix} = \begin{bmatrix} K_\rho {}^r e_x \\ K_\rho {}^r e_y \\ K_\alpha e_\theta \end{bmatrix}, \quad (11)$$

where ${}^r \mathbf{u}$ represents the velocity command expressed in the local frame of the robot. The linear velocities ${}^r v_x$ and ${}^r v_y$ are computed separately from the corresponding error. The same gain K_ρ was used for both as a simplification. This reduces the number of parameters to be tuned, although it does not mean that the longitudinal and lateral responses are identical. The orientation is corrected separately through the gain K_α .

The implementation used a position tolerance of 0.01 m. When the magnitude of the position error was less than or equal to that value, ${}^r v_x$ and ${}^r v_y$ were set to zero to avoid very small linear commands near the target position, while the angular correction remained active.

The general logic of the loop, shown in Figure 2, can be summarized as follows:

- the desired pose enters as the reference;
- this reference is compared with the current estimated pose;
- the difference between the two generates an error;
- the error feeds the proportional controller;
- the controller generates the desired chassis velocity;

- inverse kinematics transforms this desired velocity into target velocities for the wheels;
- the robot moves and a new pose estimate is obtained;
- this new pose goes back to the error computation, closing the loop.

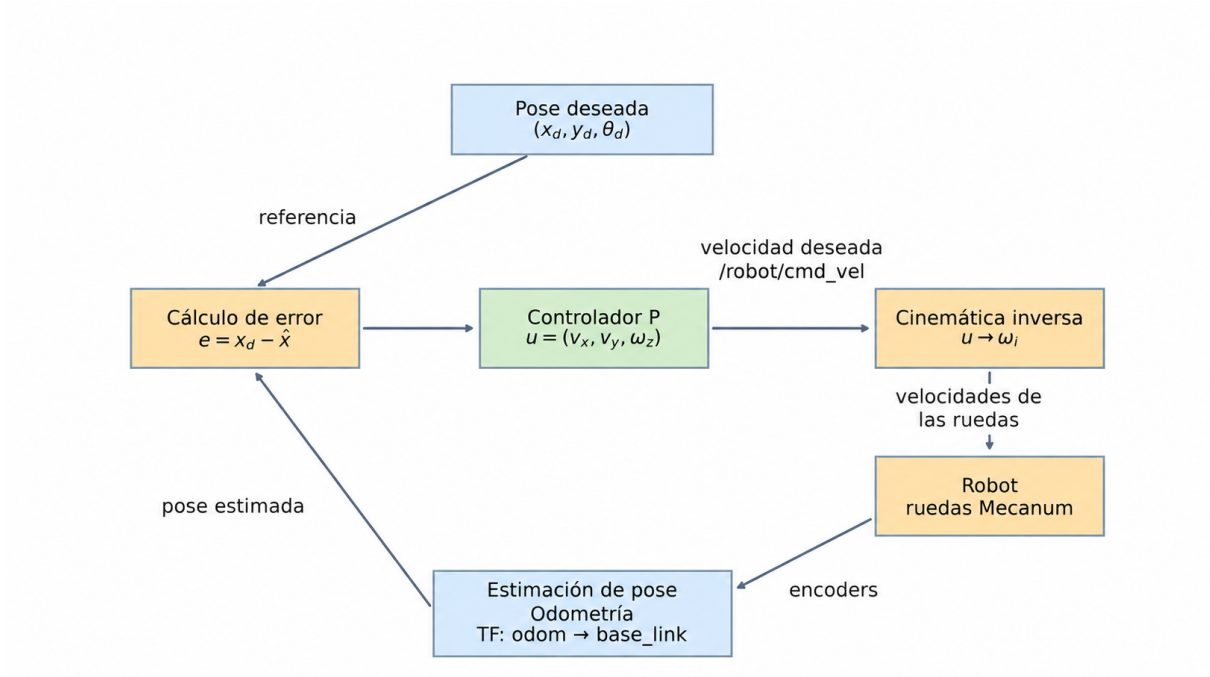


Figure 2: General scheme of closed-loop control for pose tracking using odometric feedback.

4.3 Trajectory tracking

In trajectory tracking, the reference is not a single fixed pose but a sequence of target poses. At each instant, the controller finds the closest point of the trajectory and selects as target another point located approximately at the distance defined by the *lookahead*. As the robot advances, the reference also advances along the path. With a lower value, the robot tries to stay closer to the path; with a higher value, it takes points farther ahead and may cut the corners.

This allows the robot not only to converge to a final pose, but also to follow a complete path. The performance of the system therefore depends on two aspects: the ability of the controller to correct the error and the quality of the pose estimate used as feedback. If the odometry accumulates too much error, the controller may generate corrections based on an estimated pose that no longer matches the actual pose of the robot.

5 EKF-based localization

To improve the pose estimate, a localization stage based on an extended Kalman filter (EKF) was added. It combines two sources of information: the motion prediction obtained from the kinematic model and the odometry, and the observations of landmarks in the environment, whose positions are known in advance in the map.

5.1 State and motion model

The filter state represents the pose of the robot in the plane, expressed in the global frame w

$${}^w\mathbf{x}_t = \begin{bmatrix} {}^wx_t \\ {}^wy_t \\ {}^w\theta_t \end{bmatrix}.$$

In this expression, wx_t and wy_t represent the position of the robot in the global frame, while ${}^w\theta_t$ represents the orientation of the robot frame r with respect to the global frame w .

The model input is given by the velocities obtained from the odometry, expressed in the local frame of the robot

$${}^r\mathbf{u}_t = \begin{bmatrix} {}^rv_{x,t} \\ {}^rv_{y,t} \\ {}^r\omega_{z,t} \end{bmatrix}.$$

The components ${}^rv_{x,t}$ and ${}^rv_{y,t}$ correspond to the linear velocities in the robot frame, while ${}^r\omega_{z,t}$ represents the rotation about the vertical axis.

The prediction is not considered exact. To represent its uncertainty, a process noise is added

$$\mathbf{w}_t \sim \mathcal{N}(\mathbf{0}, Q),$$

so that the model can be written as

$${}^w\mathbf{x}_t = f({}^w\mathbf{x}_{t-1}, {}^r\mathbf{u}_t) + W_t\mathbf{w}_t.$$

Without this noise, the motion model of the platform is

$${}^wx_t = {}^wx_{t-1} + ({}^rv_{x,t} \cos({}^w\theta_{t-1}) - {}^rv_{y,t} \sin({}^w\theta_{t-1})) \Delta t, \quad (12)$$

$${}^wy_t = {}^wy_{t-1} + ({}^rv_{x,t} \sin({}^w\theta_{t-1}) + {}^rv_{y,t} \cos({}^w\theta_{t-1})) \Delta t, \quad (13)$$

$${}^w\theta_t = \text{norm}({}^w\theta_{t-1} + {}^r\omega_{z,t} \Delta t). \quad (14)$$

The function $\text{norm}(\cdot)$ maps the orientation to the interval $[-\pi, \pi]$, preventing two equivalent angles from being represented by values that differ by a full turn.

To propagate the covariance, the Jacobian of the motion model with respect to the state is also needed

$$F_t = \begin{bmatrix} 1 & 0 & (-{}^r v_{x,t} \sin({}^w \theta_{t-1}) - {}^r v_{y,t} \cos({}^w \theta_{t-1})) \Delta t \\ 0 & 1 & ({}^r v_{x,t} \cos({}^w \theta_{t-1}) - {}^r v_{y,t} \sin({}^w \theta_{t-1})) \Delta t \\ 0 & 0 & 1 \end{bmatrix}.$$

In the implementation, the process noise is added directly to the three state components, so

$$W_t = I_3.$$

The errors in x , y and θ were considered separately, so Q was taken as diagonal. In addition, the same value was used for x and y as a simplification, without treating one direction as more uncertain than the other

$$Q = \text{diag}(0.001, 0.001, 0.0004).$$

In terms of standard deviation, these values correspond to approximately 3.16 cm for x and y , and to 0.02 rad, about 1.15° , for θ .

5.2 Landmark detection

The detector processes the LiDAR scans. First, it discards the measurements that are outside the valid range of the sensor. The remaining points are converted from polar to Cartesian coordinates and transformed from the sensor frame to the robot frame, **base_link**, using the full TF. After this transformation, $z = 0$ is set and zero-length vectors are discarded. The rest of the processing is carried out in the xy plane of the robot.

Consecutive measurements are then grouped as long as the distance between them in the xy plane is smaller than 0.10 m, a value that matches the known diameter of the posts. When the separation reaches or exceeds that value, the current group is considered finished and a new one starts.

For each group, the point closest to the robot in that plane is selected. Neither the mean nor the geometric centroid of the points is computed. Since the selected point corresponds approximately to the visible surface of the post, its center is estimated by shifting it 0.05 m, equal to the post radius, in the direction from the robot to the post. The resulting position is expressed by the distance to the estimated center of the post in the plane and the angle that gives its direction with respect to the x axis of the robot. This angle is computed with $\text{atan2}(y, x)$. The published point cloud also has $z = 0$ in **base_link**.

5.3 Observation model

The landmark measurements are expressed in polar coordinates with respect to the robot frame

$${}^r\mathbf{z}_t = \begin{bmatrix} \rho \\ \phi \end{bmatrix}.$$

In this expression, ρ is the distance between the robot and the landmark, while ϕ is the observation angle measured with respect to the longitudinal axis of the chassis. The noise of these measurements is represented by

$$\mathbf{v}_t \sim \mathcal{N}(\mathbf{0}, R),$$

so the observation model can be written as

$${}^r\mathbf{z}_t = h({}^w\mathbf{x}_t, {}^w\mathbf{m}) + V_t \mathbf{v}_t,$$

where ${}^w\mathbf{m} = ({}^w m_x, {}^w m_y)^T$ represents the known position of the landmark in the map. From the robot pose and that position, the expected measurement is computed as

$$\rho = \sqrt{({}^w m_x - {}^w x)^2 + ({}^w m_y - {}^w y)^2}, \quad (15)$$

$$\phi = \text{atan2}({}^w m_y - {}^w y, {}^w m_x - {}^w x) - {}^w \theta. \quad (16)$$

To write the Jacobian more compactly, the following are defined

$$\delta_x = {}^w m_x - {}^w x, \quad \delta_y = {}^w m_y - {}^w y, \quad q = \delta_x^2 + \delta_y^2.$$

With this notation, the Jacobian of the observation model with respect to the state is

$$H_t = \begin{bmatrix} -\frac{\delta_x}{\sqrt{q}} & -\frac{\delta_y}{\sqrt{q}} & 0 \\ \frac{\delta_y}{q} & -\frac{\delta_x}{q} & -1 \end{bmatrix}.$$

In the implementation, the noise directly affects the two components of the measurement, so

$$V_t = I_2.$$

The range and angle errors were also considered separately, so R was taken as diagonal

$$R = \text{diag}(0.001, 0.00007615435).$$

The first value corresponds to a range standard deviation of

$$\sigma_\rho = \sqrt{0.001} \approx 0.0316 \text{ m} \approx 3.16 \text{ cm},$$

while the second corresponds to an angular standard deviation of

$$\sigma_\phi = \sqrt{0.00007615435} \approx 0.00873 \text{ rad} \approx 0.5^\circ.$$

These values were chosen for the simulation tests, considering the level of precision expected for the range and angle measurements. If the filter were applied to the real robot, they would need to be retuned from measurements of the system.

5.4 Association of measurements with landmarks

Before using a measurement to correct the pose, it is necessary to determine which landmark of the map it corresponds to. Each observation is projected from polar coordinates onto the global plane. Given a measurement (ρ, ϕ) and an estimated pose $({}^w x, {}^w y, {}^w \theta)$, the estimated Cartesian position of the landmark is computed as

$$\begin{aligned} {}^w \hat{x}_L &= {}^w x + \rho \cos({}^w \theta + \phi), \\ {}^w \hat{y}_L &= {}^w y + \rho \sin({}^w \theta + \phi). \end{aligned}$$

Once the measurement is projected, the map landmark closest to that position is searched for. The association is considered valid if that distance is smaller than the defined threshold; otherwise, the measurement is discarded. In the implementation, this threshold was set to 1.0 m. This threshold is half the minimum separation between the posts of the map, which is 2 m. Since only distances strictly smaller than the threshold are accepted, the acceptance regions do not overlap.

An incorrect association can worsen the estimate instead of correcting it, which is why the threshold discards measurements without a nearby match in the map.

5.5 EKF correction

To distinguish what happens before and after incorporating a measurement, a different notation is used at each stage. The predicted mean and covariance are denoted by $\bar{\mu}_t$ and $\bar{\Sigma}_t$, while μ_t and Σ_t represent the values obtained after the correction. The estimated pose is expressed in the global frame w , so the frame superscript is not written again in the following expressions.

During the prediction, the mean and covariance are updated according to

$$\begin{aligned} \bar{\mu}_t &= f(\mu_{t-1}, {}^r \mathbf{u}_t), \\ \bar{\Sigma}_t &= F_t \Sigma_{t-1} F_t^T + W_t Q W_t^T. \end{aligned}$$

Since $W_t = I_3$ in the implementation, the covariance prediction becomes

$$\bar{\Sigma}_t = F_t \Sigma_{t-1} F_t^T + Q.$$

When a valid association is found, the received measurement is compared with the one expected from the predicted pose. The difference between the two is computed as

$$\mathbf{y}_t = {}^r \mathbf{z}_t - h(\bar{\mu}_t).$$

The second component of this difference corresponds to the observation angle, so it is normalized to the interval $[-\pi, \pi]$

$$y_{\phi,t} = \text{norm}(z_{\phi,t} - h_{\phi}(\bar{\mu}_t)).$$

Then the matrix S_t and the Kalman gain are computed

$$\begin{aligned} S_t &= H_t \bar{\Sigma}_t H_t^T + V_t R V_t^T, \\ K_t &= \bar{\Sigma}_t H_t^T S_t^{-1}. \end{aligned}$$

Since $V_t = I_2$, the first expression reduces to

$$S_t = H_t \bar{\Sigma}_t H_t^T + R.$$

This expression combines the uncertainty of the predicted pose and the uncertainty assigned to the measurement. From both, the Kalman gain determines how much the estimate should change with the received observation.

Finally, the mean and covariance are corrected through

$$\begin{aligned} \mu_t &= \bar{\mu}_t + K_t \mathbf{y}_t, \\ \Sigma_t &= (I - K_t H_t) \bar{\Sigma}_t. \end{aligned}$$

With the values used for Q and R , after each prediction and before incorporating the new observations, the predicted uncertainty expressed in the measurement variables equals or exceeds the uncertainty assigned to the sensor. This is because the prediction keeps the previous uncertainty propagated through the term $F_t \Sigma_{t-1} F_t^T$, which is positive semidefinite, and adds the contribution of Q to it.

Projecting that contribution through the Jacobian of the observation model gives 0.001 m^2 for the range, equal to the corresponding measurement variance. This value does not depend on the direction to the post because the same process noise was used for x and y . For the angle, the contribution to the variance, expressed in rad^2 , is $0.0004 + 0.001/d^2$, where d is the distance to the post in meters. The first term alone is already about 5.25 times the angular variance assigned to the sensor, regardless of the observation distance.

These values therefore justify giving more weight to the observations when correcting the prediction.

The logs make it possible to illustrate the effect of a correction. In one update of the 4 m EKF run, at $t = 0.05$ s, the estimated variance of x went from 0.00164235 m^2 before incorporating a valid observation to 0.00100697 m^2 after that correction. The reduction was approximately 38.69% in that update and shows the contribution of that observation to the estimate of x .

The filter was initialized with the pose

$$\mu_0 = (-2, 2, \pi/2)^T,$$

expressed in the `map` frame, and with the initial covariance

$$\Sigma_0 = \text{diag}(0.1, 0.1, 0.1).$$

Figure 3 summarizes how the information sources used by the system are related. The encoders provide the odometric prediction, while the LiDAR and the known map of posts provide the corrections. The ground truth is not used by the EKF and serves only to evaluate the results in the simulation.



Figure 3: Information flow used to estimate and evaluate the robot pose.

6 Experimental results

The runs in this section were carried out with the scene included in the submission, with the chassis and wheels aligned. In the localization and tracking tests, a single laser TF was used, published by the scene.

6.1 Validation of the kinematic model

To evaluate the kinematic model, four sequences of target velocities were generated and sent to the robot through the `/robot/cmd_vel` topic. Each sequence was run independently and in open loop, resetting the scene so that the robot always started from the same pose. First, the components v_x , v_y and ω_z were analyzed separately, and then the two linear velocities were combined. The switch from one step to the next was triggered by elapsed time, not by distance traveled.

The actual velocities shown in Figures 4, 5, 6 and 7 were computed from the time evolution of the pose published on `/robot/ground_truth`. This pose is expressed in the `odom` frame and describes the motion of `base_link_gt`. The velocities from `/robot/odometry` were not used as the actual reference because they are obtained from the encoders through the same forward kinematics that was being evaluated. The `twist` field of `/robot/ground_truth` was not used either, since in the recorded runs it was not consistent with the evolution of the pose. The ground truth was used as a reference independent of the odometric model.

To obtain these velocities, the jumps of θ between π and $-\pi$ were corrected first. Then the position and orientation were differentiated numerically. Central differences were used for the interior samples

$$\dot{x}_k^{gt} \approx \frac{x_{k+1}^{gt} - x_{k-1}^{gt}}{t_{k+1} - t_{k-1}}, \quad \dot{y}_k^{gt} \approx \frac{y_{k+1}^{gt} - y_{k-1}^{gt}}{t_{k+1} - t_{k-1}}.$$

The angular velocity was computed in the same way

$$\omega_{z,k}^{gt} \approx \frac{\tilde{\theta}_{k+1}^{gt} - \tilde{\theta}_{k-1}^{gt}}{t_{k+1} - t_{k-1}},$$

where $\tilde{\theta}^{gt}$ represents the orientation after correcting the jumps between π and $-\pi$.

To compare these velocities with the setpoints sent through `/robot/cmd_vel`, the linear components obtained in the global frame were transformed to the local frame of the robot through

$$\begin{bmatrix} v_x^{gt} \\ v_y^{gt} \end{bmatrix}_k = \begin{bmatrix} \cos \theta_k^{gt} & \sin \theta_k^{gt} \\ -\sin \theta_k^{gt} & \cos \theta_k^{gt} \end{bmatrix} \begin{bmatrix} \dot{x}^{gt} \\ \dot{y}^{gt} \end{bmatrix}_k.$$

Since the ground truth was recorded at approximately 20 Hz, consecutive samples were about 0.05 s apart. For the interior samples, the previous and the next sample were used,

so each velocity estimate covered an interval of approximately 0.10 s. At the ends, second-order one-sided differences were used. No additional smoothing was applied, so part of the small oscillations observed may be due to the numerical differentiation. This processing was used only to generate the validation plots and played no role in the odometry or the control.

In sequences 2 and 4, the nominal duration of each step was kept at 0.5 s to prevent the accumulated displacement from taking the robot to the posts of the scene. With this duration, each step kept about ten samples to analyze the response. The following tables show the nominal durations; the effective intervals between the first command and the stop were 10.10 s, 5.05 s, 8.05 s and 5.55 s for sequences 1 to 4, respectively, since some steps lasted an extra 0.05 s.

The lower panels of the four figures show the error between the setpoint and the actual response of the simulator, defined as

$$e_q(t) = q_{\text{target}}(t) - q_{\text{actual}}(t), \quad q \in \{v_x, v_y, \omega_z\}.$$

The direct comparison between the pose estimated by odometry and the ground truth is presented at the end of this subsection.

6.1.1 Sequence 1: velocity along the x axis

Table 1: Sequence 1. Target velocities along the x axis.

	Sequence 1									
Δt [s]	1	1	1	1	1	1	1	1	1	1
v_x [m/s]	-0.1	-0.2	-0.3	-0.4	-0.5	-0.6	-0.7	-0.8	-0.9	-1
v_y [m/s]	0	0	0	0	0	0	0	0	0	0
ω_z [rad/s]	0	0	0	0	0	0	0	0	0	0

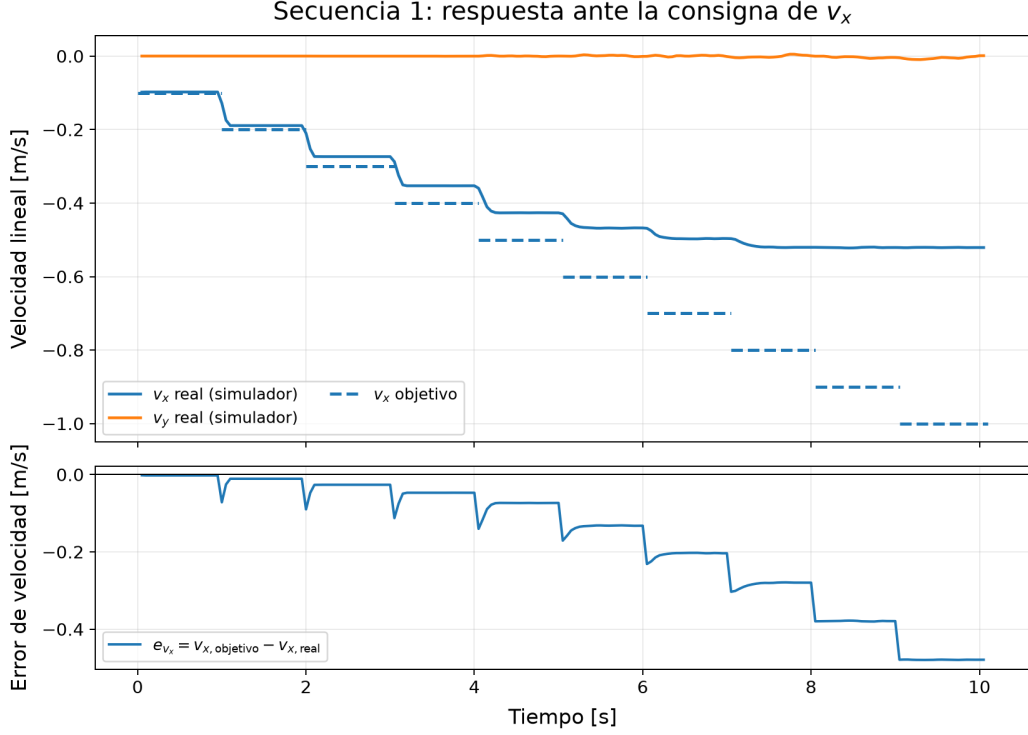


Figure 4: Robot response to setpoints of increasing magnitude in sequence 1.

Longitudinal motion was evaluated with the target velocities in Table 1, increasing in magnitude, while v_y and ω_z were kept at zero. For the smaller setpoints, the actual velocity roughly follows the reference. As the requested value increases, the actual velocity starts to move away from the reference and v_x settles near -0.52 m/s, although the setpoint continues up to -1 m/s.

The lower panel shows how the magnitude of the error grows as velocities of larger magnitude are requested and the actual velocity stays near that limit. Throughout the test, v_y stays close to zero, so the longitudinal motion does not produce a significant lateral component.

6.1.2 Sequence 2: velocity along the y axis

Table 2: Sequence 2. Target velocities along the y axis.

	Sequence 2									
Δt [s]	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5
v_x [m/s]	0	0	0	0	0	0	0	0	0	0
v_y [m/s]	-0.1	-0.2	-0.3	-0.4	-0.5	-0.6	-0.7	-0.8	-0.9	-1
ω_z [rad/s]	0	0	0	0	0	0	0	0	0	0

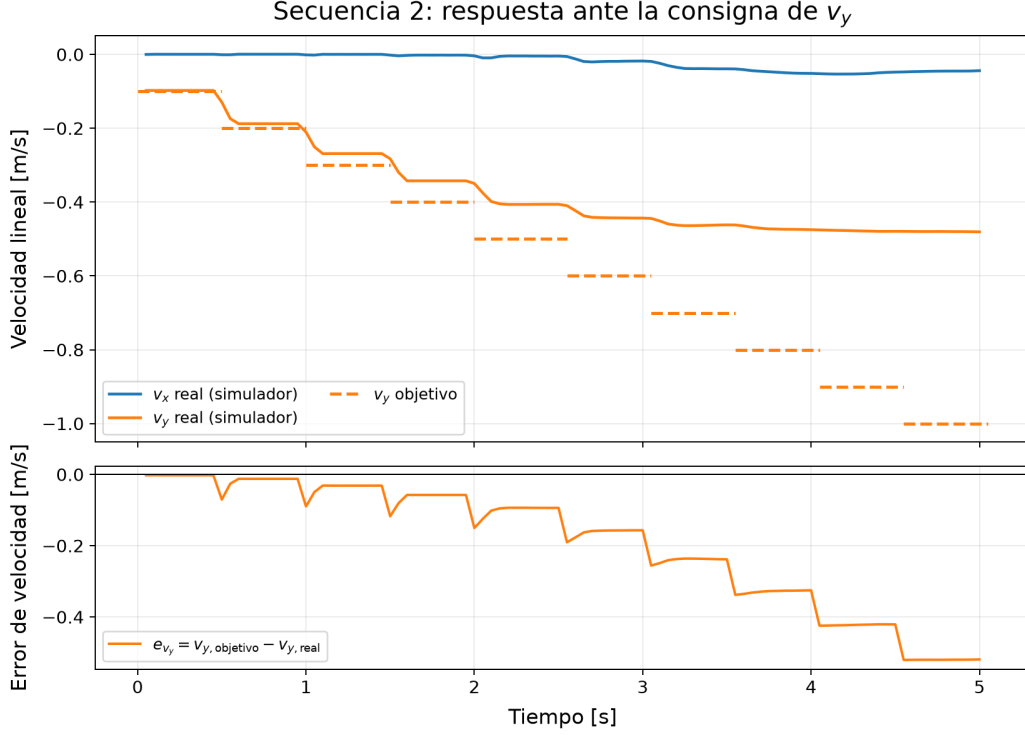


Figure 5: Robot response to setpoints of increasing magnitude in sequence 2.

Lateral motion, with the target velocities in Table 2, behaves similarly to longitudinal motion, although the limit appears at a somewhat lower magnitude. For small references, the actual velocity stays close to the setpoint. Between approximately 2.5 s and 3 s the difference becomes clearer and v_y settles near -0.48 m/s, while the reference keeps increasing in magnitude.

For the highest lateral setpoints, a small component also appears in v_x , even though its reference stays at zero. In addition to the limit observed in v_y , there is then a small longitudinal motion that had not been requested.

6.1.3 Sequence 3: angular velocity

Table 3: Sequence 3. Target angular velocities.

	Sequence 3						
Δt [s]	1	1	1	1	1	1	2
v_x [m/s]	0	0	0	0	0	0	0
v_y [m/s]	0	0	0	0	0	0	0
ω_z [rad/s]	0.3	0.6	0.9	1.2	1.5	1.8	2.1

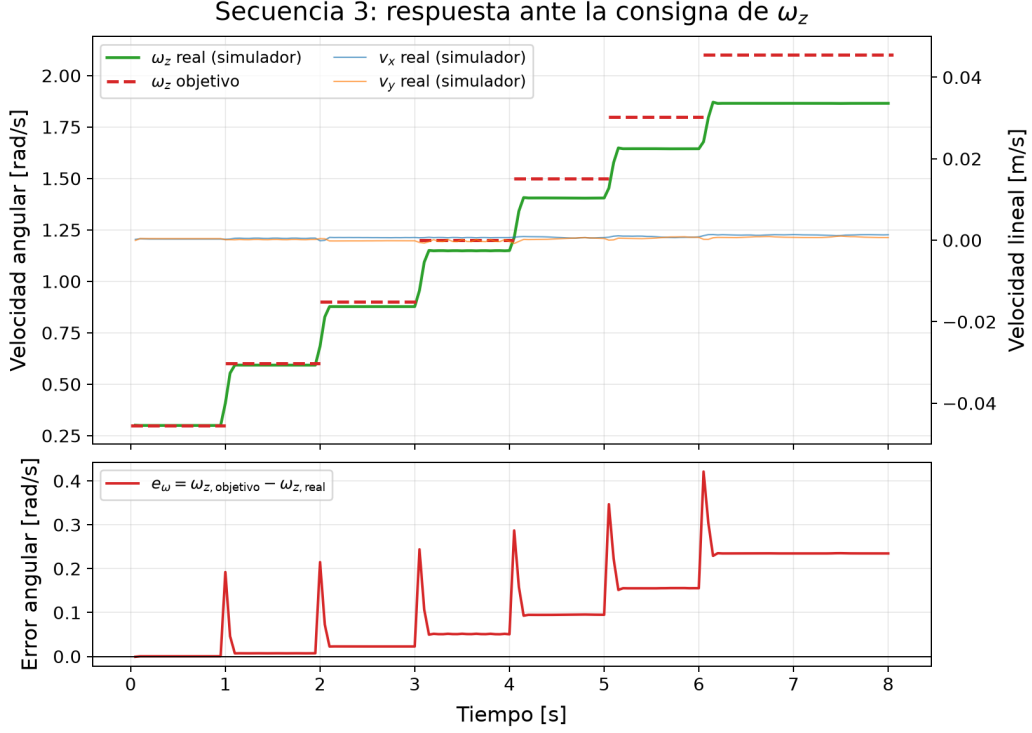


Figure 6: Robot response to increasing target angular velocities in sequence 3.

For the target velocity steps in Table 3, the actual angular velocity follows the reference and keeps increasing over the whole range tested. For the highest setpoints, however, it falls progressively below the reference. With the last requested value, 2.1 rad/s, the robot reaches approximately 1.87 rad/s.

The error peaks coincide with the step changes, because the actual velocity does not reach the new reference immediately. After each transition, the difference decreases, although in the last steps it does not fully return to zero. At the same time, v_x and v_y stay close to zero, so the rotation does not produce a significant linear component.

6.1.4 Sequence 4: combined linear velocities

Table 4: Sequence 4. Simultaneous target velocities in v_x and v_y .

	Sequence 4										
Δt [s]	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5
v_x [m/s]	0.1	0.2	0.3	0.4	0.5	0.5	0.4	0.3	0.2	0.1	-0.5
v_y [m/s]	0.1	0.2	0.3	0.4	0.5	0.5	0.5	0.5	0.5	0.5	0.5
ω_z [rad/s]	0	0	0	0	0	0	0	0	0	0	0

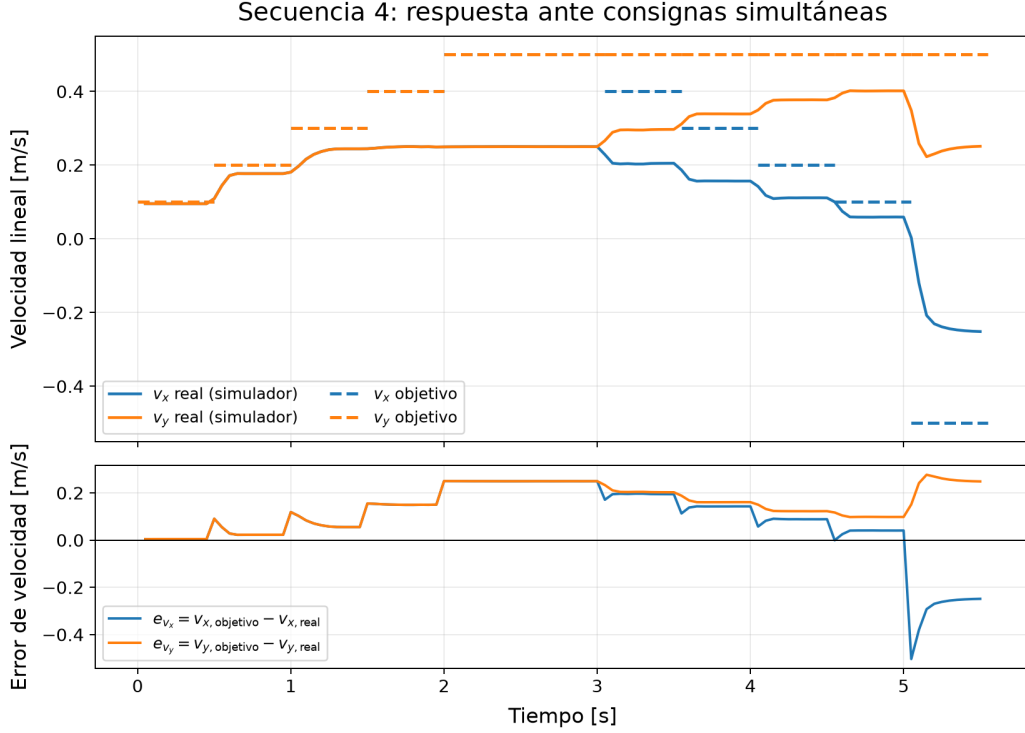


Figure 7: Robot response to simultaneous target velocities in sequence 4.

In the sequence of Table 4, the references for v_x and v_y increase together from $+0.1$ m/s to $+0.5$ m/s. Since both setpoints coincide during this first part, their dashed lines overlap in the plot. For the lowest values, the two actual velocities roughly follow the reference. The difference becomes more visible when about 0.3 m/s or more is requested on both axes.

When $v_x = v_y = +0.5$ m/s is requested, the actual responses stay near $+0.25$ m/s for each component. Afterwards, the v_y reference remains at $+0.5$ m/s, while the v_x reference decreases from $+0.4$ m/s to $+0.1$ m/s. As the requested velocity in x decreases, the actual lateral velocity increases to approximately $+0.40$ m/s.

Only in the last interval does the v_x setpoint become negative. In that segment, $v_x = -0.5$ m/s and $v_y = +0.5$ m/s are requested: the two setpoints have the same magnitude but opposite signs. The actual responses reach approximately $v_x = -0.25$ m/s and $v_y = +0.25$ m/s. Since the error is obtained by subtracting the actual velocity from the target velocity, the component corresponding to v_x also takes a negative value during this segment.

This behavior suggests that the simulated platform reaches a limit when both linear components are requested at the same time. The velocities reached when testing each axis separately are not maintained in the same way when both motions are combined.

6.1.5 Comparison between odometry and ground truth

To complete the validation, the difference between the pose estimated by odometry and the actual pose published by the simulator was analyzed. The same runs from sequences 1, 2 and 3 were used, corresponding to longitudinal motion, lateral displacement and

rotation. In the velocity plots, the ground truth pose was differentiated to obtain the actual velocities; at this stage, the two poses are compared directly.

Since the odometry and the ground truth were not published at exactly the same instants, the common time interval was considered and the odometric pose was interpolated at the ground truth times. The intervals $[0.15, 10.70]$ s, $[0.15, 5.65]$ s and $[0.15, 8.65]$ s were used for sequences 1, 2 and 3, respectively, which include the 0.5 s of logging after the stop. In the linear tests, the position error was computed as the Euclidean distance between the two positions. For the rotation, the absolute value of the angular difference normalized between $-\pi$ and π was used. The results are presented in Figure 8.

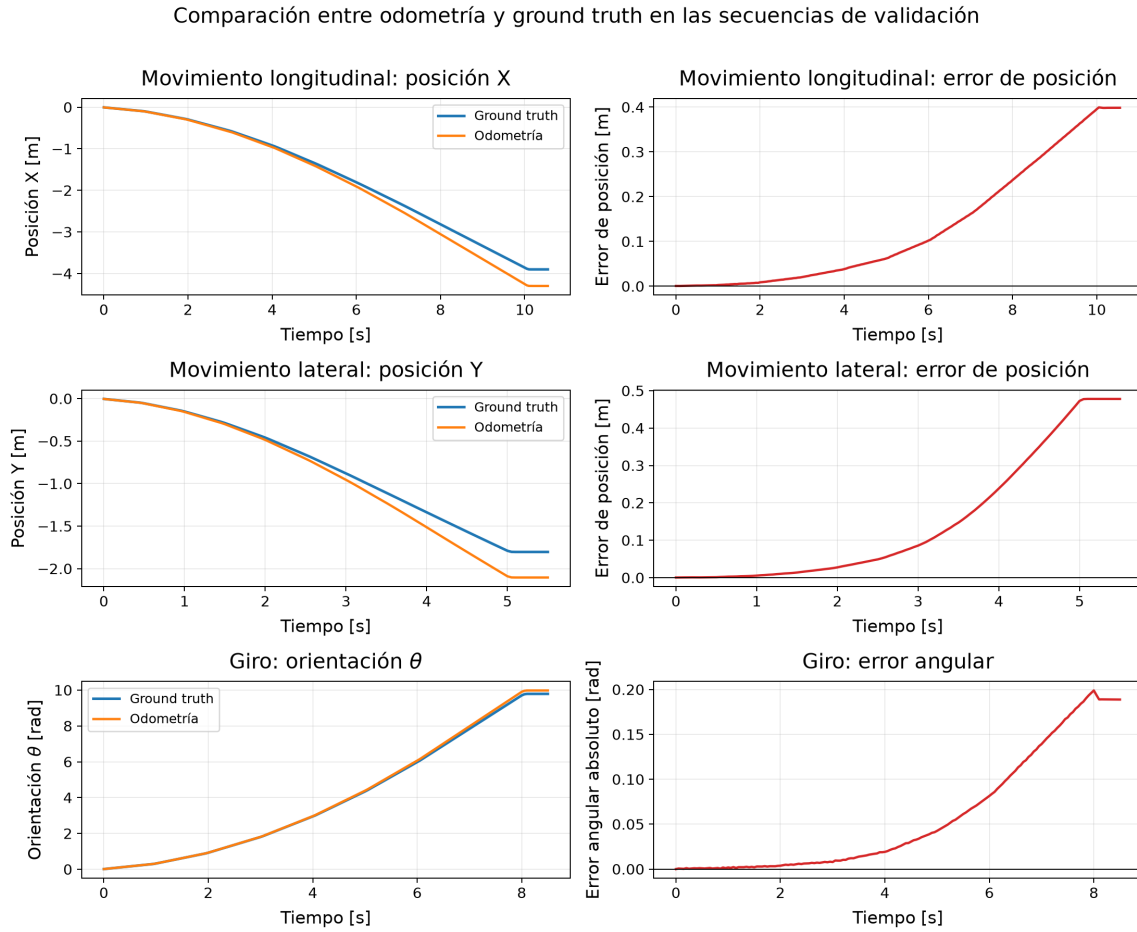


Figure 8: Comparison between the pose estimated by odometry and the ground truth in the longitudinal, lateral and angular motion sequences.

In the longitudinal motion, the odometry follows the evolution of the position in x , although the difference grows during the run. The mean position error was 0.130 m, the maximum 0.399 m and the final 0.398 m. The lateral motion shows similar behavior in the y position, with a mean error of 0.149 m, a maximum of 0.478 m and a final value of 0.478 m.

During the rotation, the odometric orientation also follows the trend of the ground truth, but the angular difference grows as the test progresses. The mean absolute angular error was 0.058 rad, the maximum 0.199 rad and the final 0.189 rad. The three tests show that the odometry reproduces the general trend of the motion, although the small errors in the

velocities obtained from the encoders accumulate when they are integrated to estimate the pose.

6.2 Controller tuning: comparison of configurations

After validating the kinematic model, closed-loop trajectory tracking was evaluated on an exact square with 4 m sides, centered at (0, 0) m in `map`, equivalent to (-2, -2) m in `odom`, and traversed clockwise. In the latter frame, its vertices are (0, 0), (0, -4), (-4, -4) and (-4, 0), in meters. The reference was built from four straight segments and consisted of 801 points, including the final point that closes the path. Its discretization does not set the actual duration of the runs, since the controller moves between targets according to the *lookahead* criterion.

At each point, a desired orientation pointing from the center of the square toward that point was defined. During the tests, the controller used as feedback the pose estimated by odometry, corresponding to the `odom` $\rightarrow$ `base_link` transformation. To evaluate the result, the actual pose published by CoppeliaSim on `/robot/ground_truth` was used, associated with the `odom` $\rightarrow$ `base_link_gt` transformation. The ground truth was used only to generate the plots and compute the metrics, and played no role in the control action. The velocities shown for each experiment correspond to the commands published by the controller on `/robot/cmd_vel`.

To evaluate the position, the distance between the ground truth and the closest point of the desired trajectory was computed. This geometric definition does not depend on the robot and the reference traversing the square with the same timing. The path error was expressed as

$$e_{path}(k) = \sqrt{(x_k - x_{path,k}^*)^2 + (y_k - y_{path,k}^*)^2}. \quad (17)$$

In Equation 17, (x_k, y_k) is the position obtained from the ground truth and $(x_{path,k}^*, y_{path,k}^*)$ is the closest point of the discretized reference trajectory. All the points of the discretized reference were used, instead of only the main waypoints, so that their spacing would not affect the measurement.

The angular error was computed by comparing the ground truth orientation with the radial orientation assigned to the geometrically closest point of the discretized trajectory. This reference is different from the target pose selected through the *lookahead*; therefore, the metric does not represent the internal angular error of the controller or a localization error. The difference was normalized between $-\pi$ and π before computing its absolute value.

The plots and error metrics include the samples from $t = 0$ up to the stop command of each run. The reported duration is measured from the first motion command, at $t = 0.05$ s, up to that stop.

The parameter K_ρ is associated with the correction of the position error, and K_α with the angular correction. Strictly speaking, the *lookahead* is not a constant of the proportional controller, but it defines how far ahead along the trajectory the target pose is taken, so it also affects the way the robot follows the trajectory.

To limit the number of runs, experiment 1 was taken as the baseline. Starting from that configuration, one parameter was changed at a time, testing a lower and a higher value, while the other two were kept fixed.

The following values were considered:

$$\begin{aligned} K_\rho &\in \{0.2, 0.5, 0.9\}, \\ K_\alpha &\in \{0.6, 1, 1.4\}, \\ lookahead &\in \{0.2, 0.5, 1\}. \end{aligned}$$

From these values, the configurations summarized in Table 5 were defined:

Table 5: Configurations tested for the controller.

Experiment	K_ρ	K_α	<i>lookahead</i>
exp1	0.5	1.0	0.5
exp2	0.2	1.0	0.5
exp3	0.9	1.0	0.5
exp4	0.5	0.6	0.5
exp5	0.5	1.4	0.5
exp6	0.5	1.0	1.0
exp7	0.5	1.0	0.2

The linear and angular velocities sent to the robot in the seven experiments are presented in Appendix A, in Figures 33 and 34. The time comparison between the actual pose and the target selected by the controller is presented in Appendix B.

Comparación de trayectorias para distintas configuraciones de K_ρ

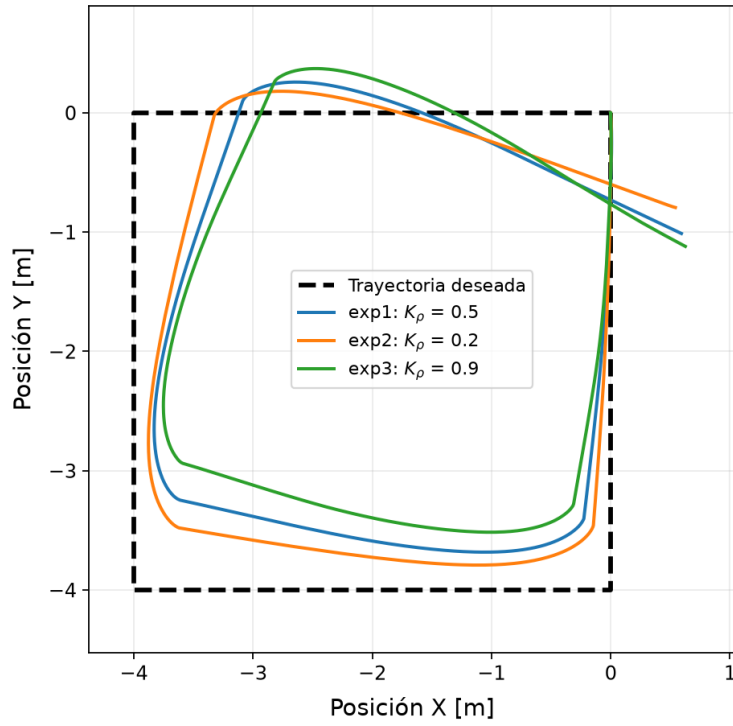


Figure 9: Effect of K_ρ on trajectory tracking.

Comparación de trayectorias para distintas configuraciones de K_α

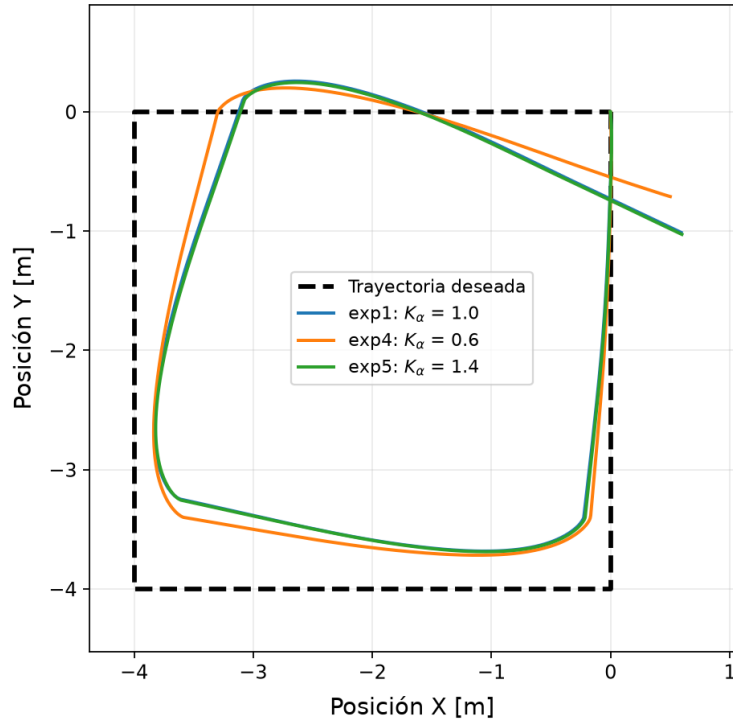


Figure 10: Effect of K_α on trajectory tracking.

Comparación de trayectorias para distintas configuraciones de lookahead

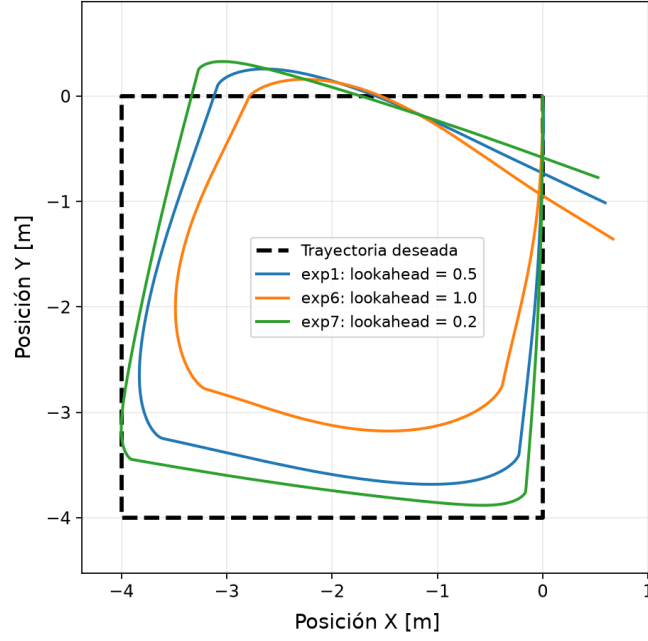


Figure 11: Effect of the *lookahead* on trajectory tracking.

Figures 9, 10 and 11 show how the shape of the path changes when each parameter is modified. The cut sides and rounded corners correspond to the response of the robot and not to the desired trajectory.

Varying K_ρ produces clear differences. With 0.2, the robot cuts the sides of the square less. With 0.9, on the other hand, it moves farther from the reference and ends farther from the closing point. Experiment 1 lies between the two: a stronger correction does not necessarily improve the shape of the tracking.

The trajectories with $K_\alpha = 1.0$ and $K_\alpha = 1.4$ practically overlap. With $K_\alpha = 0.6$, there are differences at the changes of direction and a smaller separation from the reference on the last side.

The *lookahead* changes the path more clearly: with 0.2 m, corner cutting is reduced, while with 1.0 m a large part of the square is cut and a shorter trajectory is completed. In both cases, some separation from the reference remains at the end.

The path errors obtained with the different configurations are shown in Figures 12, 13 and 14.



Figure 12: Path error for different K_ρ configurations.

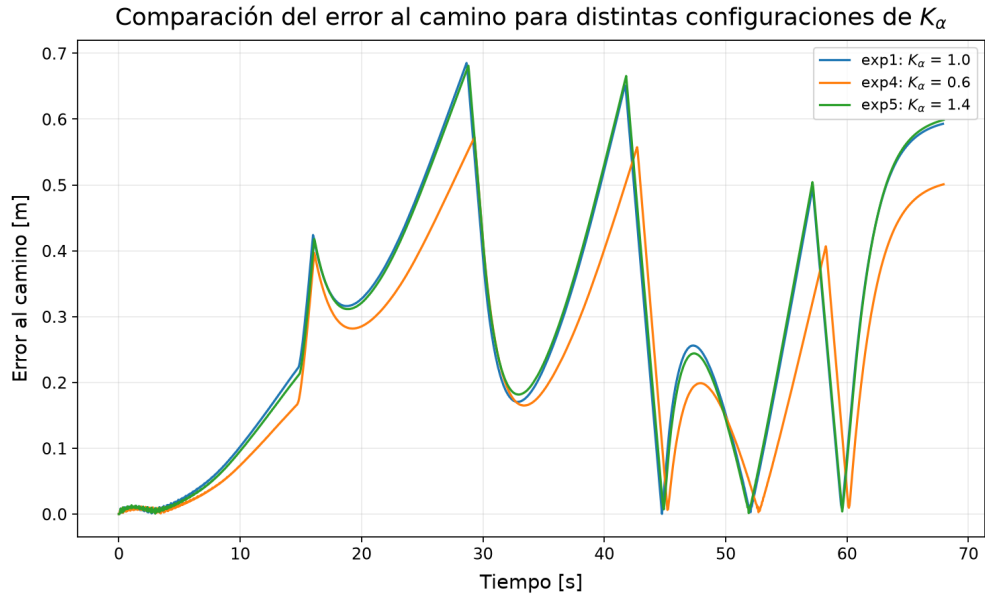


Figure 13: Path error for different K_α configurations.

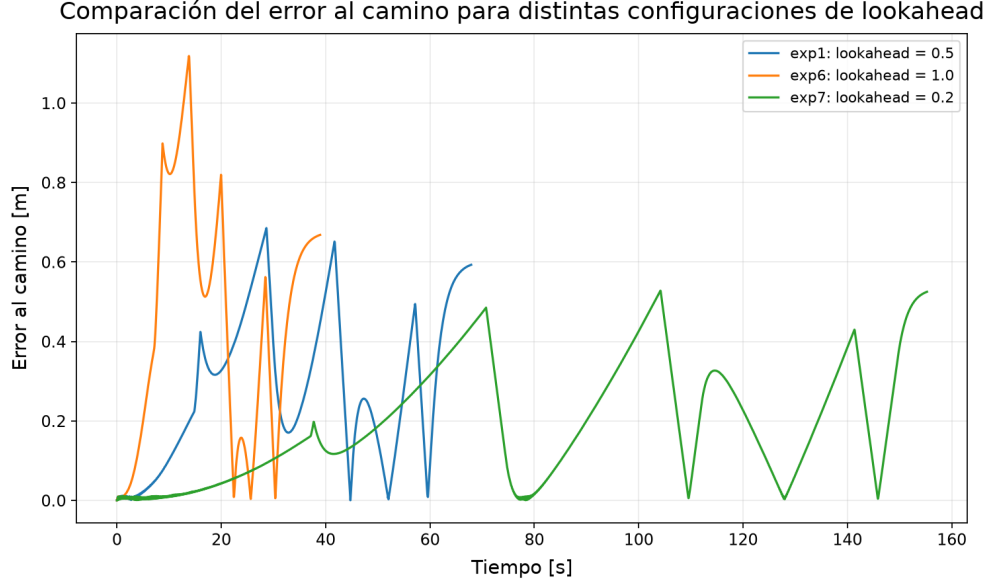


Figure 14: Path error for different *lookahead* configurations.

The error plots show increases at the segment changes, but also differences that persist along the sides and at the end of the run. With $K_p = 0.9$, higher peaks appear and the error stays high during a good part of the run. Among the K_α variants, the value 0.6 reduces the main peaks compared with 1.0 and 1.4. The *lookahead* of 1.0m produces the highest peak, while with 0.2m the error reaches approximately 0.527 m.

Table 6: Path error metrics for the control experiments.

Experiment	K_p	K_α	<i>lookahead</i>	Mean error [m]	Max. error [m]	Final error [m]	Time [s]
exp1	0.5	1.0	0.5	0.286	0.685	0.593	67.85
exp2	0.2	1.0	0.5	0.227	0.542	0.542	163.80
exp3	0.9	1.0	0.5	0.348	0.905	0.626	40.50
exp4	0.5	0.6	0.5	0.242	0.573	0.501	67.90
exp5	0.5	1.4	0.5	0.287	0.681	0.599	67.90
exp6	0.5	1.0	1.0	0.454	1.118	0.668	38.90
exp7	0.5	1.0	0.2	0.194	0.527	0.525	155.20

Table 6 shows the trade-off between path error and travel time. Experiment 7 had the lowest mean and maximum errors, although it needed more than twice the time of experiment 1. Experiment 6 gave the shortest time, but also the highest mean, maximum and final errors. Experiment 2, in turn, was the slowest. The lowest final error corresponded to experiment 4, so reducing the mean and maximum errors did not guarantee ending closer to the path.

K_p and the *lookahead* were the parameters that changed the travel time the most, in addition to affecting the path errors. The variations of K_α also had an effect on the position: with $K_\alpha = 0.6$, the mean, maximum and final errors decreased compared with experiment 1, with practically the same duration. Increasing the gain to 1.4 did not produce a comparable improvement.

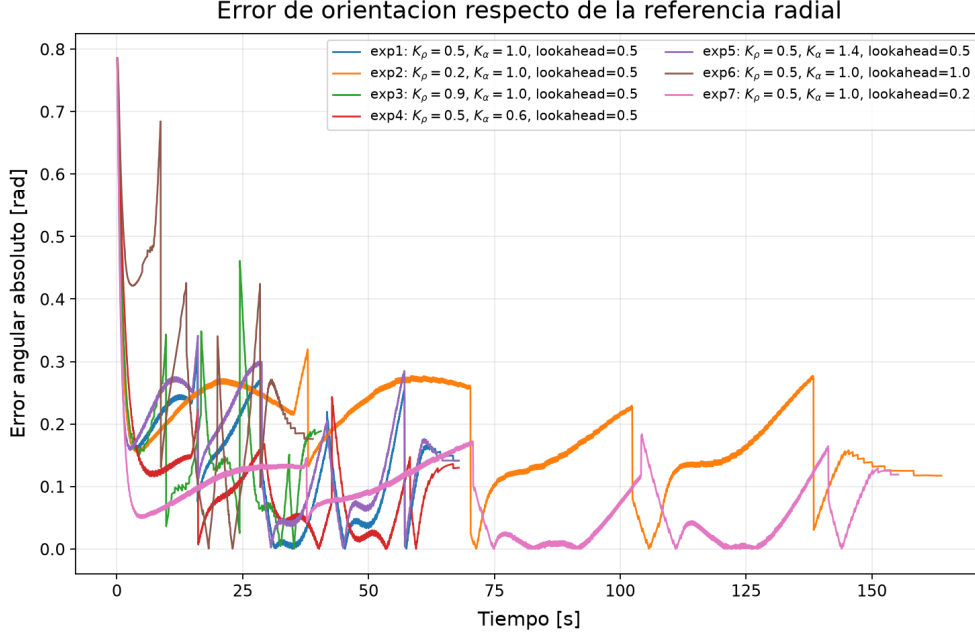


Figure 15: Absolute angular error with respect to the orientation associated with the closest point of the desired trajectory.

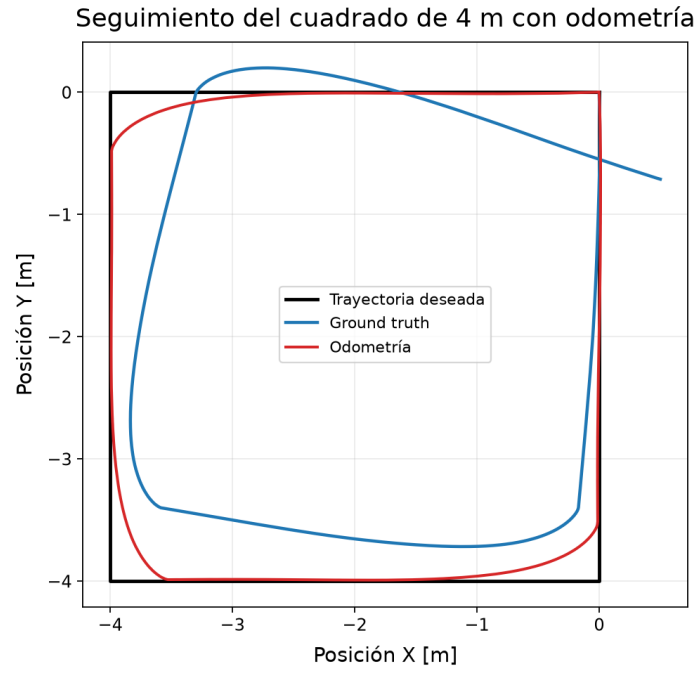
Figure 15 complements the position metrics. For experiments 1, 4 and 5, the mean angular errors were approximately 0.142 rad, 0.103 rad and 0.162 rad, respectively. Among these configurations, the lowest value was obtained with $K_\alpha = 0.6$, which also reduced the path errors. The maximum angular error was practically the same in all seven runs and was dominated by the initial difference from the radial orientation; the differences between configurations are better seen in the later evolution and in the mean error.

No configuration was best on all metrics. For the final comparisons, the configuration selected from experiment 4 was kept, with $K_p = 0.5$, $K_\alpha = 0.6$ and a *lookahead* of 0.5 m. It had lower path errors than experiment 1 and the lowest mean angular error among the three configurations evaluated for K_α , and it completed the path in 67.90 s.

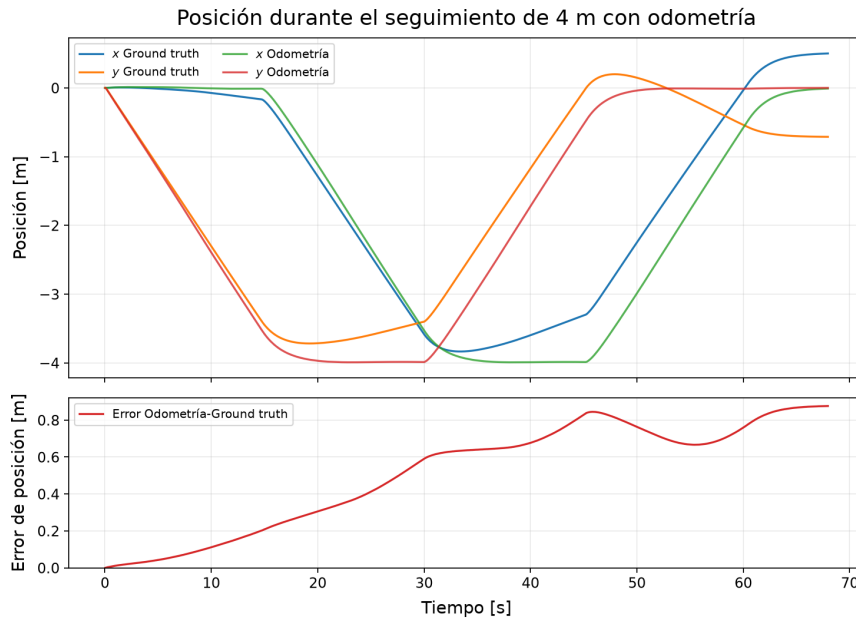
Experiment 7 achieved lower mean and maximum path errors, although it needed more than twice the time, while experiment 6 was faster but cut the trajectory considerably. Odometric feedback and EKF-based feedback were compared using the same parameters.

6.2.1 Detailed analysis of the selected configuration

To analyze the tracking in more detail, the logs from experiment 4 of the previous comparison were used. The controller used the odometric pose as feedback. The plots and localization errors were evaluated between $t = 0$ and $t = 67.95$ s, while the motion commands spanned 67.90 s.

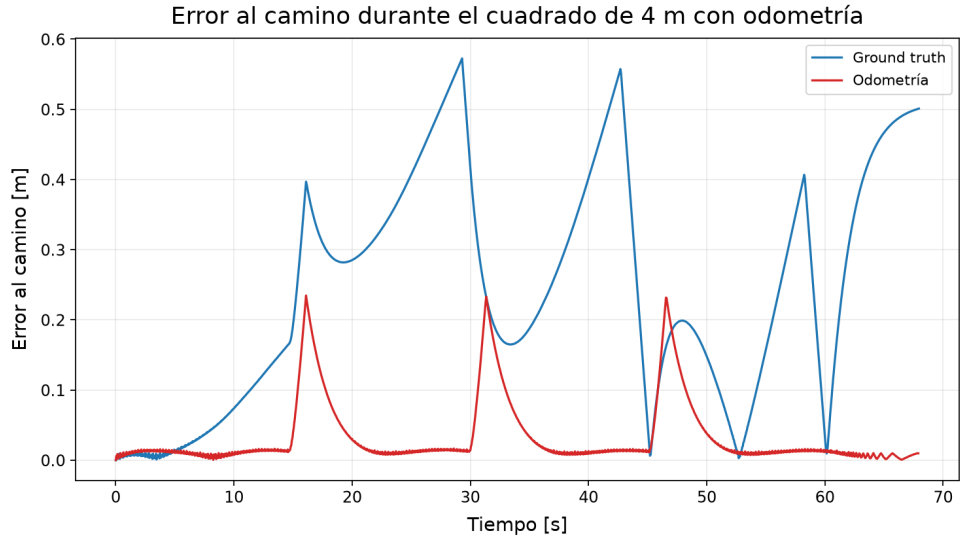


(a) Desired trajectory, ground truth and odometry.

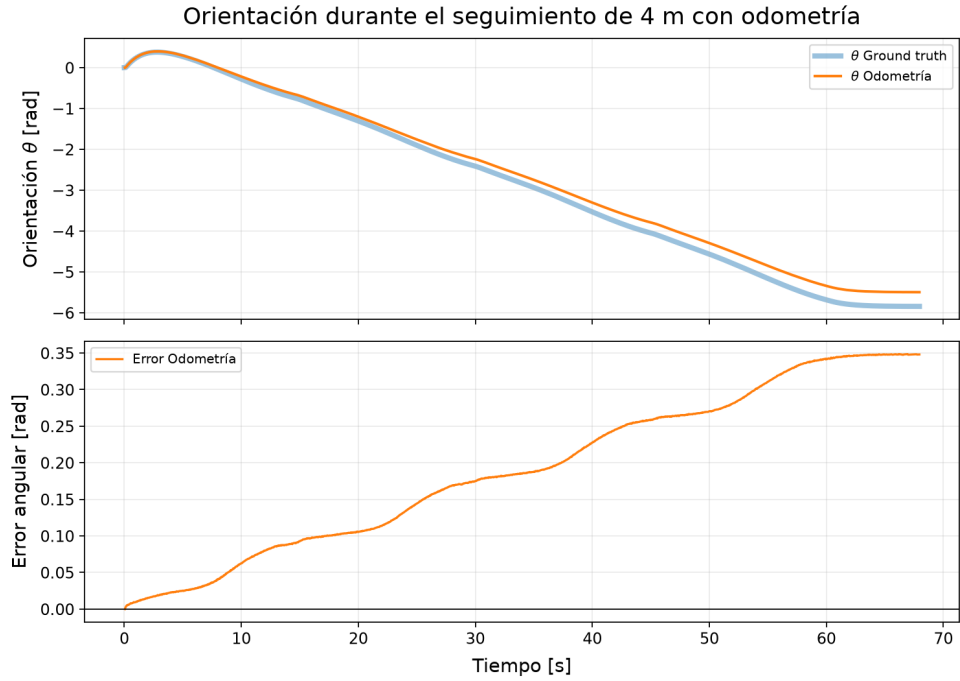


(b) Time evolution of the position and direct error of the odometry with respect to the ground truth.

Figure 16: Results of tracking an exact square with 4 m sides using odometric feedback.



(c) Path error computed with ground truth and with odometry.



(d) Odometric orientation and angular error with respect to the ground truth.

Figure 16: Results of tracking an exact square with 4m sides using odometric feedback (continued).

In the trajectory panel of Figure 16, the reference keeps the four straight sides and the corners of the square. The odometry stays close to that reference, with rounded corners. The ground truth, by contrast, cuts the first sides and moves away from the last one, without returning to the closing point. In the plot of position over time, the difference between the odometry and the ground truth becomes more visible during the second half of the run and at the end.

The lower panel of the position plot shows the direct error between the position estimated by odometry and the position reported by the ground truth. Since the two signals were

not published at exactly the same instants, the odometric pose was interpolated at the times of the ground truth samples. The error was defined as

$$e_{\text{pos}}(t) = \sqrt{(x_{\text{odom}}(t) - x_{\text{gt}}(t))^2 + (y_{\text{odom}}(t) - y_{\text{gt}}(t))^2}.$$

The difference between the two positions had a mean value of 0.512 m, reached a maximum of 0.876 m and ended at that same value. Although the curve shows a partial reduction in the second half of the run, it grows again toward the end: the odometry follows the general motion of the robot, but accumulates a difference with respect to its actual position.

The direct position error compares the odometry and the ground truth at the same instant. The path error, instead, measures the distance between each position and the closest point of the desired trajectory. Thus, the path error panel makes it possible to compare the actual tracking error with the one computed from the odometry.

According to the odometry, the path error decreases after the side changes and ends at 0.010 m, but the one computed with the ground truth stays high during a good part of the run and ends at 0.501 m. This difference shows that the error computed with the pose used by the controller can be small, even when the robot is far from the path.

In orientation, the odometric curve follows the general evolution of the ground truth, but the separation between the two grows during the run. The error panel shows growth in stages and a difference that persists when the robot stops. The odometry makes it possible to close the loop in this test, but it accumulates a difference in both position and orientation as the run progresses.

6.3 Manual tests of EKF-based localization

To evaluate localization independently of trajectory tracking, three manual runs were carried out. None of them used the trajectory generator or the trajectory controller. The commands were published directly on `/robot/cmd_vel` through an auxiliary ROS2 script. The tests included a circular motion, a square traversed through longitudinal and lateral motions, and an in-place rotation. In this way, the filter was evaluated under the different motions that the platform allows.

In all three cases, the odometry and the pose estimated by the EKF were compared with the ground truth. Before the comparisons, the EKF pose was transformed to the `odom` frame. To compare the signals instant by instant, only the common time interval within the command window of each test was considered. The plots show the relative time from the start of that window. The last sample of each repeated timestamp was kept, and the odometry and EKF pose were interpolated at the ground truth times. The position error corresponds to the distance between each estimate and the ground truth. For the orientation, the ground truth orientation was subtracted from the estimated one and the difference was normalized between $-\pi$ and π . The plots keep the sign of this difference, while the metrics summarize its absolute value.

6.3.1 Circular motion

In the first run, the input was kept at

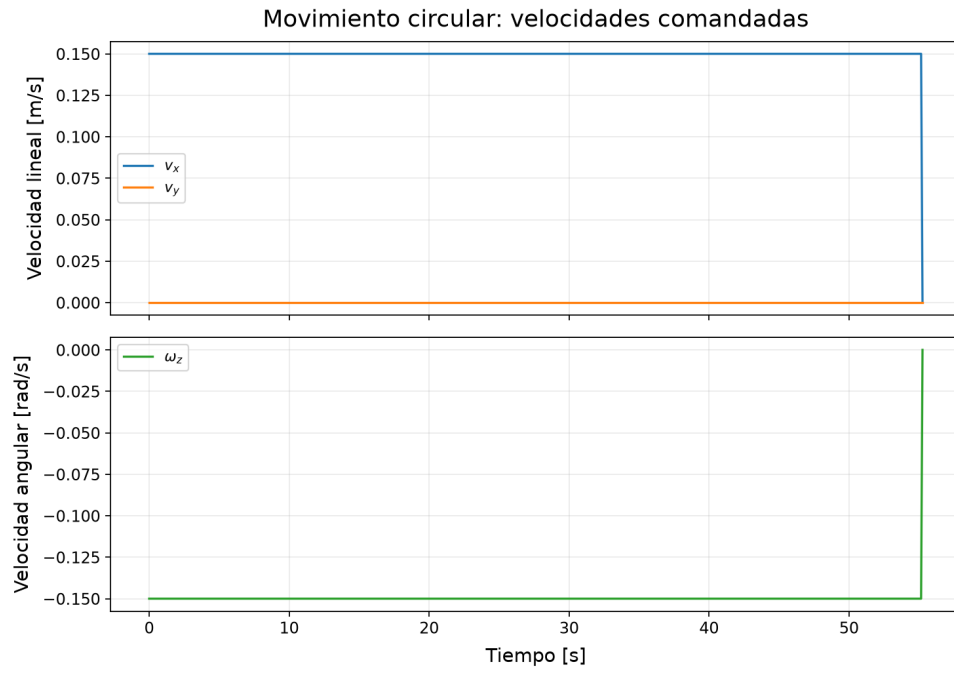
$${}^r\mathbf{u} = \begin{bmatrix} 0.15 \\ 0.00 \\ -0.15 \end{bmatrix},$$

where the first two components are expressed in meters per second and the third in radians per second. The configured duration was 55.2 s and the recorded one 55.25 s. The difference corresponds to the time resolution of the publisher. Since a constant longitudinal velocity and a constant angular velocity were combined, with zero lateral velocity, the expected motion is approximately circular. The radius associated with these commands is

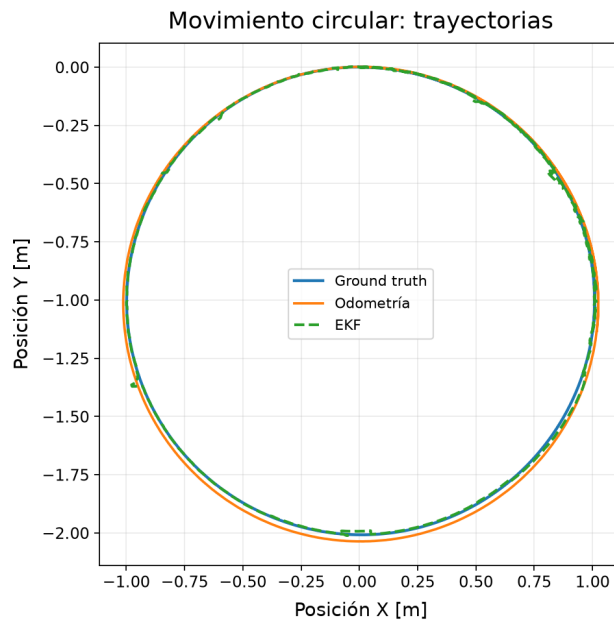
$$r_{\text{mov}} = \frac{{}^rv_x}{|{}^r\omega_z|} = \frac{0.15}{0.15} = 1 \text{ m}.$$

Integrating the commanded velocities over the 55.25 s recorded gives an expected distance of 8.2875 m and an angular change of 8.2875 rad in absolute value. The ground truth recorded 8.0189 m and 7.9857 rad, respectively. Both values were somewhat lower, but in similar proportions, so the ratio between the distance traveled and the angular change was approximately 1.0042 m and the radius of the motion stayed close to 1 m.

The sequence ended by sending zero linear and angular velocity commands, without a deceleration ramp. The jump to zero seen in the plot corresponds to those commands and does not mean that the robot stops instantly.

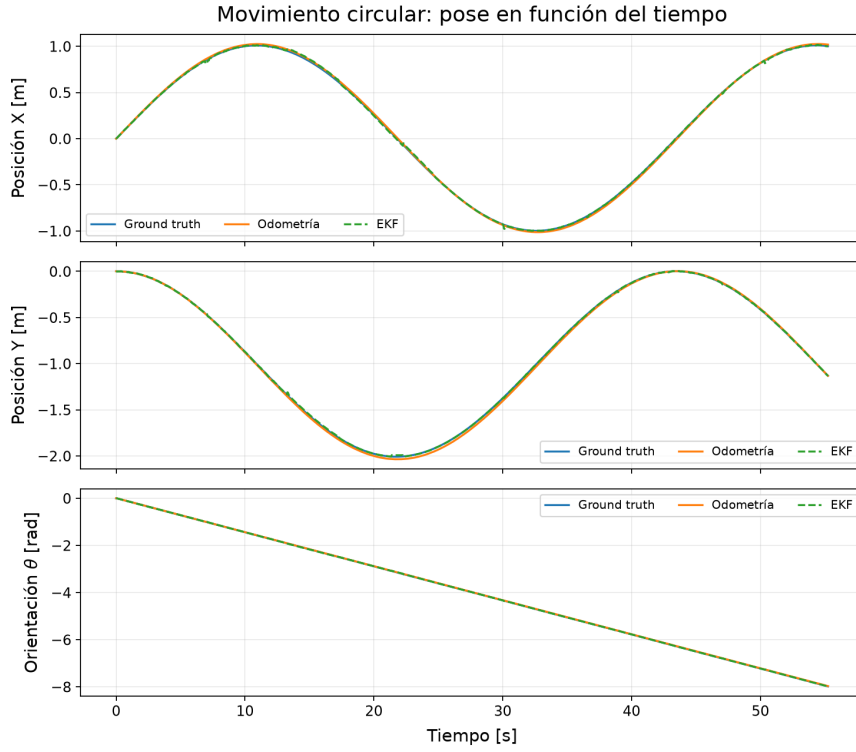


(a) Commanded velocities during the test.

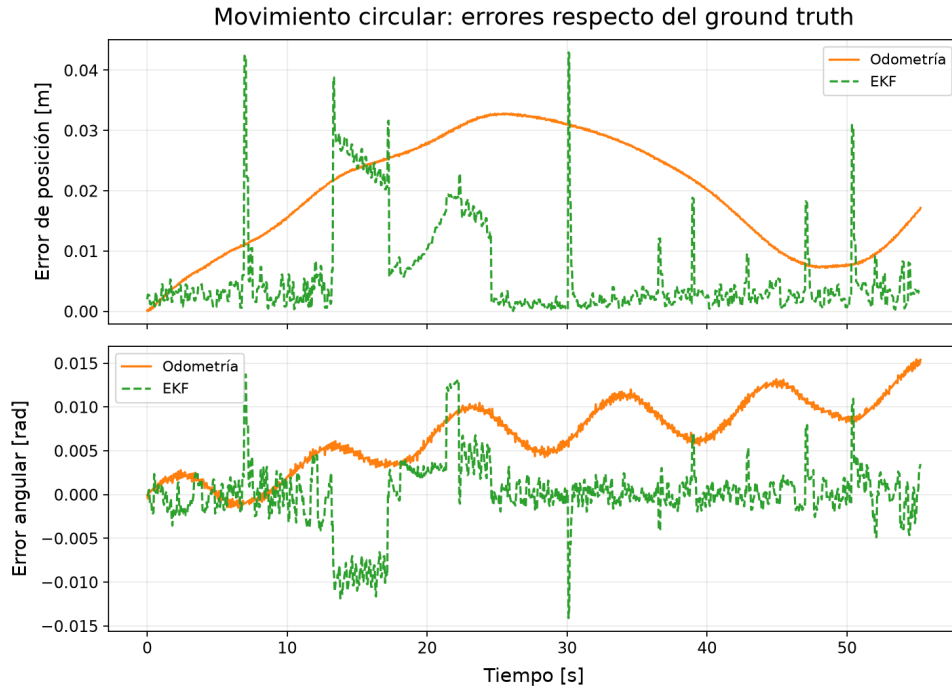


(b) Trajectories obtained from the ground truth, odometry and EKF.

Figure 17: Results of the manual circular motion test.



(c) Time evolution of the pose.



(d) Position and orientation errors with respect to the ground truth.

Figure 17: Results of the manual circular motion test (continued).

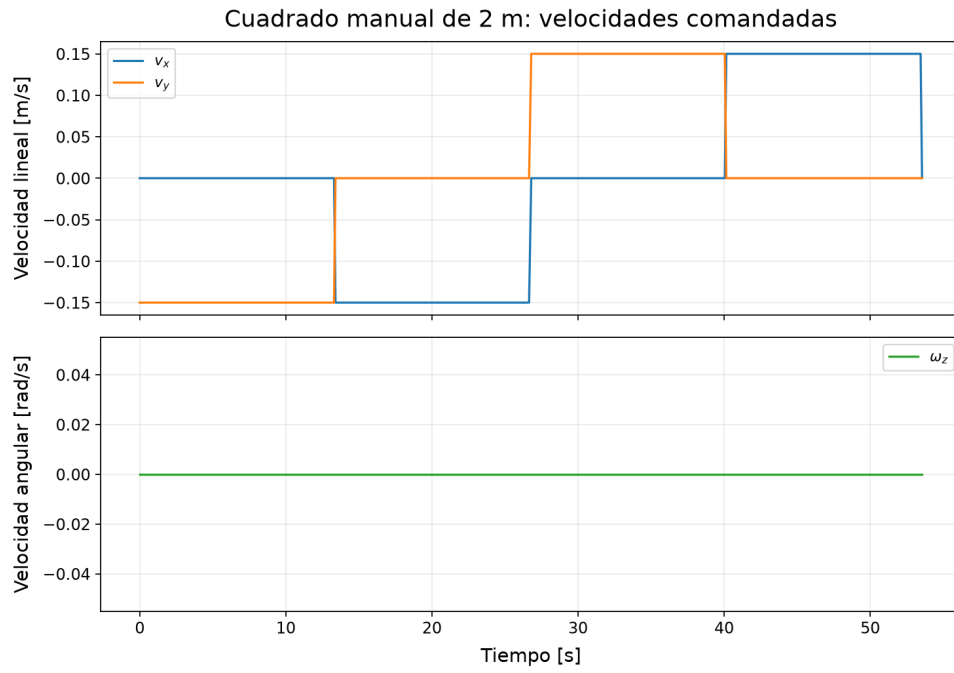
In Figure 17, the trajectory estimated by the EKF generally stays closer to the ground truth than the odometry. The mean position error decreased from 0.0193 m to 0.0061 m and the final error from 0.0172 m to 0.0029 m. However, the EKF maximum was higher: it reached 0.0430 m, compared with 0.0328 m for the odometry.

In orientation, the EKF reduced all three absolute errors: mean, maximum and final. The mean decreased from 0.0068 rad to 0.0023 rad and the final from 0.0154 rad to 0.0035 rad. Although the angular estimate also shows isolated peaks, its maximum stayed below that of the odometry. The filter improved the result during a good part of the run, but it was not better at every instant.

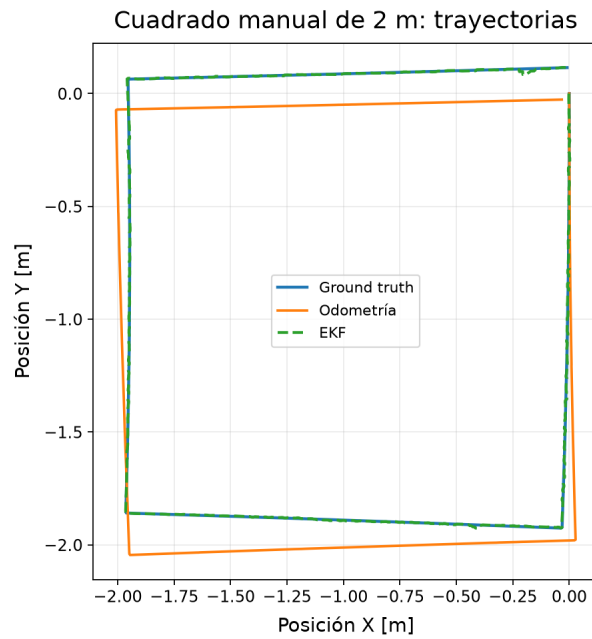
6.3.2 Manual square with longitudinal and lateral motions

The second run was designed to traverse a square with 2 m sides, keeping the angular velocity at zero. The path was carried out through four successive commands: first $v_y = -0.15$ m/s, then $v_x = -0.15$ m/s, then $v_y = +0.15$ m/s and finally $v_x = +0.15$ m/s. In each segment, the other linear component was kept at zero.

The configured duration per side was approximately 13.33 s, while the total recorded duration was 53.55 s. The 2 m correspond to the distance computed from the setpoint and its nominal duration, since in this test there was no feedback to ensure the position at the end of each segment.



(a) Commanded velocities during the test.



(b) Trajectories obtained from the ground truth, odometry and EKF.

Figure 18: Results of the manual test on a square with 2 m sides.

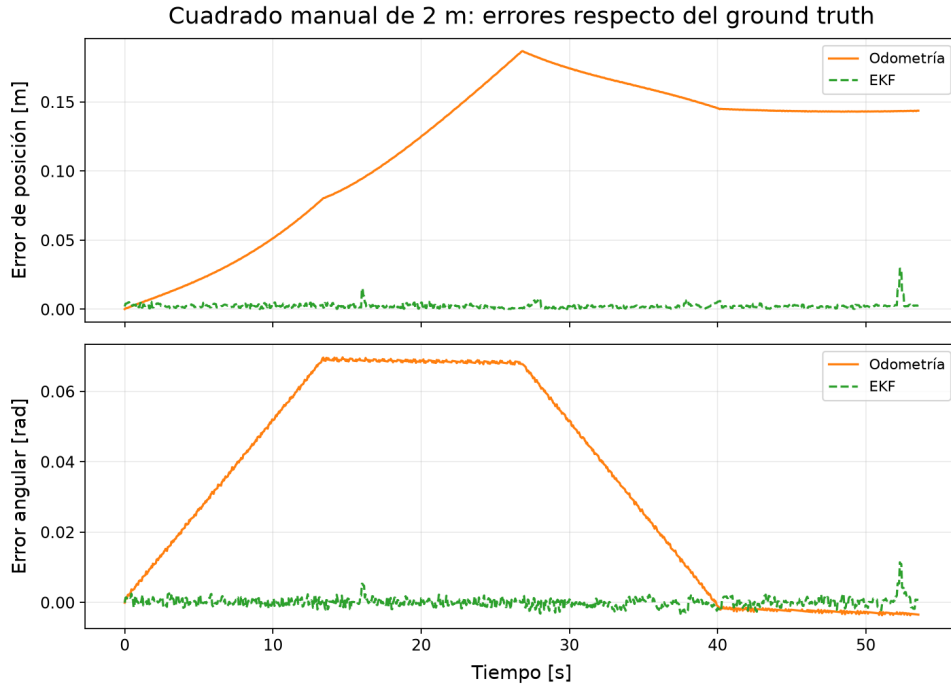
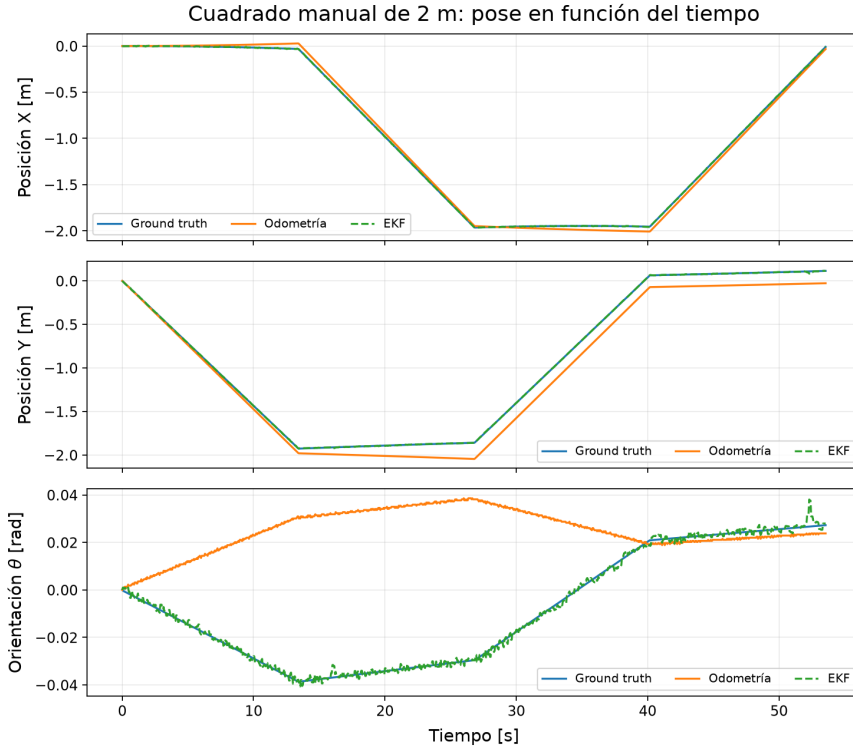


Figure 18: Results of the manual test on a square with 2m sides (continued).

The manual square, presented in Figure 18, shows the clearest difference between the two estimates. The odometry accumulates a separation that grows during the first segments and does not disappear at the end of the run. The EKF, by contrast, stays practically on top of the ground truth both in the trajectory and in the x and y components.

The mean position error decreased from 0.1174 m with odometry to 0.0023 m with the EKF, while the final error went from 0.1436 m to 0.0029 m. The improvement also appears in the orientation: the mean absolute angular error decreased from 0.0348 rad to 0.0009 rad. The mean, maximum and final errors were lower with the EKF in both position and orientation. These results show that the landmark observations corrected the drift accumulated during the longitudinal and lateral motions.

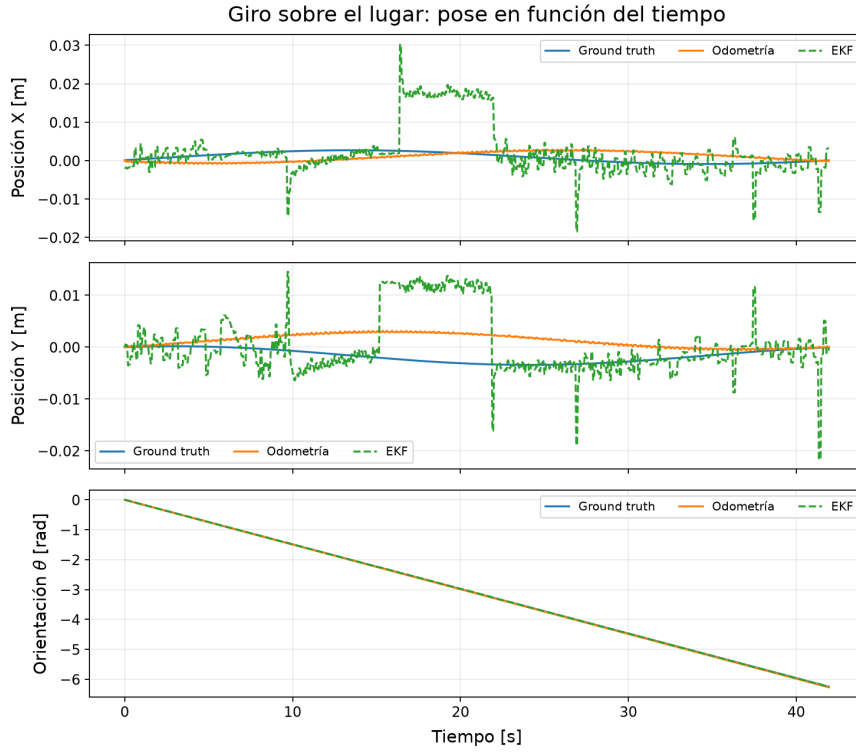
6.3.3 In-place rotation

In the third run, both linear velocities were kept at zero and a constant angular velocity of -0.15 rad/s was applied. A duration of approximately 41.89 s was configured to complete one turn with that setpoint, and the recorded interval was 41.95 s, without requesting any displacement of the chassis.

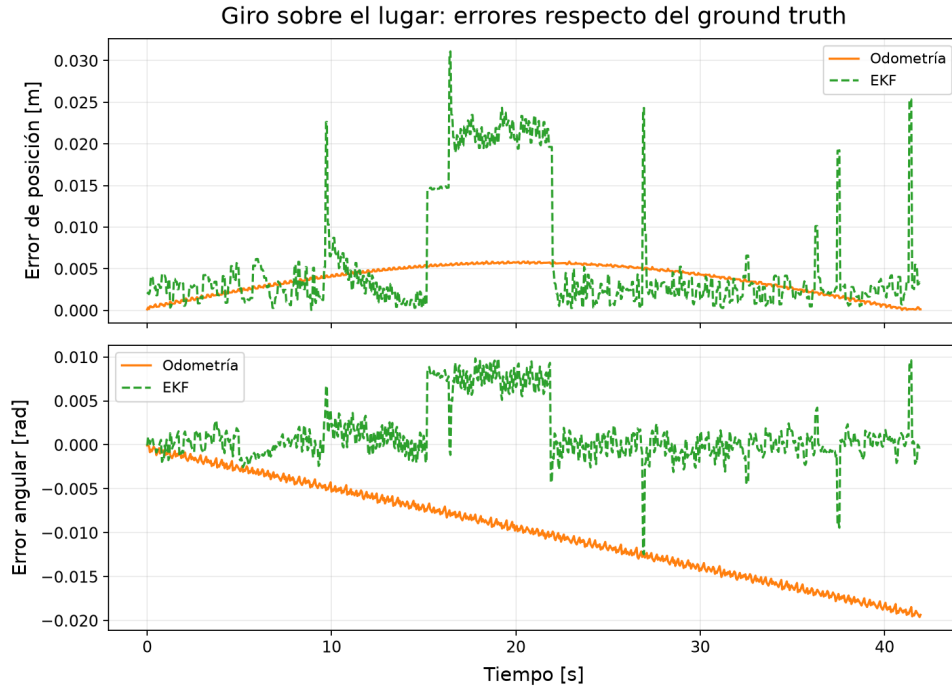


(a) Commanded velocities during the test.

Figure 19: Results of the manual in-place rotation test.



(b) Time evolution of the pose.



(c) Position and orientation errors with respect to the ground truth.

Figure 19: Results of the manual in-place rotation test (continued).

In the time evolution panel of Figure 19, the ground truth shows that the actual position stays close to the starting point throughout the run. In position, the odometry had a lower mean error than the EKF: 0.0037 m versus 0.0058 m. The EKF maximum was 0.0311 m. The jumps that appear in the x and y curves correspond to corrections of the pose

estimated by the filter and not to sudden displacements of the robot.

Since no linear motion was commanded, the odometry already had a very low position error. The EKF, on the other hand, kept updating x , y and θ from the landmark observations. The logs show that the number of associations per message changed during the rotation: there were three at the beginning, then two and, between 9.90 s and 21.90 s of the log, only one. Later, it varied again between two and four. These intervals describe the messages received and do not guarantee continuous visibility. When the set of observed posts changes, the measurements used to correct the pose also change, which can produce more noticeable adjustments.

With a single associated post, the measurement provides a distance and an angle, but it offers less information to separate the position and orientation corrections. Under these conditions, part of the measurement noise can show up in the x and y components. This may explain why the EKF estimate is more irregular than the odometry during some intervals and why, in this test, the improvement is concentrated in the orientation.

The mean absolute angular error decreased from 0.0099 rad with odometry to 0.0022 rad with the EKF, and the final error went from 0.0194 rad to 0.0003 rad. In this motion, the filter did not improve the position, but it did prevent the angular difference from continuing to accumulate.

To relate the results of the three tests to the observations used by the filter, the number of landmarks received, associated and discarded in each test was recorded.

As for the previous metrics, these counts only considered the observations recorded during the command window of each run: [0.10, 55.35] s for the circular test, [0.10, 53.65] s for the square and [0.10, 42.05] s for the rotation. The counts correspond to observations received over time, not to distinct posts of the map.

Table 7: Landmark associations during the manual localization tests.

Test	Received	Associated	Discarded	Association [%]
Circular	1100	1100	0	100.00
Square	1467	1467	0	100.00
Rotation	802	802	0	100.00

Table 7 shows that, in all three tests, 100% of the received observations fell within the threshold defined for the association. This does not guarantee that the associations are correct, that the same number of posts was observed throughout the run, or that the measurements are free of noise. This result is therefore consistent with the occasional corrections observed during the rotation.

6.3.4 Overall comparison

Figure 20 compares the mean position and orientation errors obtained in the three tests.

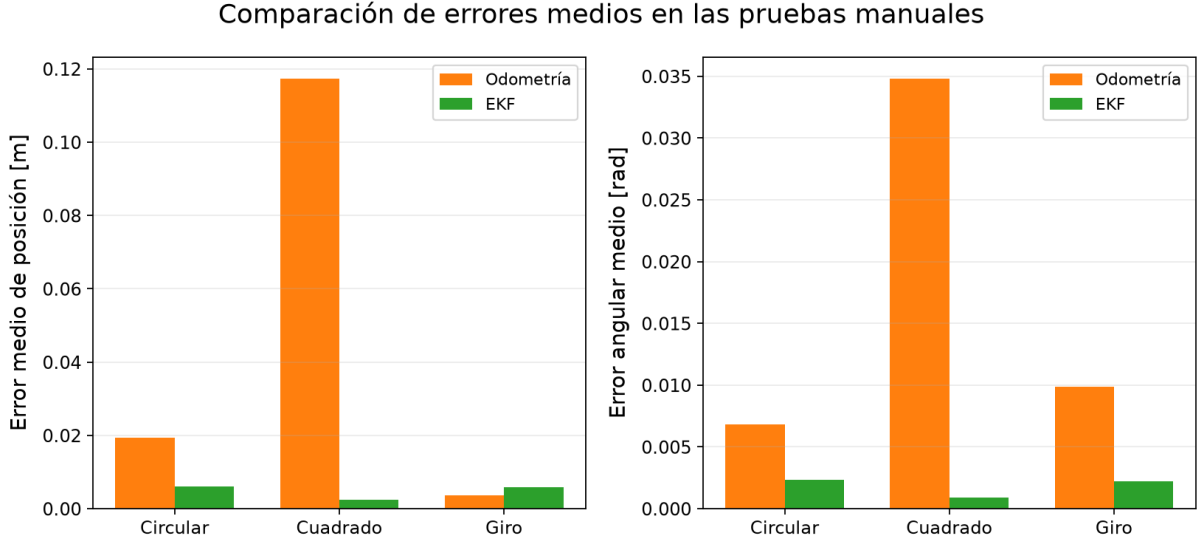


Figure 20: Comparison of the mean position and orientation errors in the three manual tests. The angular error is summarized by its absolute value.

Table 8: Error metrics of the manual localization tests.

Position error [m]				
Test	Method	Mean	Maximum	Final
Circular	Odometry	0.0193	0.0328	0.0172
Circular	EKF	0.0061	0.0430	0.0029
Square	Odometry	0.1174	0.1866	0.1436
Square	EKF	0.0023	0.0293	0.0029
Rotation	Odometry	0.0037	0.0059	0.0001
Rotation	EKF	0.0058	0.0311	0.0032
Absolute angular error [rad]				
Test	Method	Mean	Maximum	Final
Circular	Odometry	0.0068	0.0154	0.0154
Circular	EKF	0.0023	0.0141	0.0035
Square	Odometry	0.0348	0.0697	0.0035
Square	EKF	0.0009	0.0114	0.0007
Rotation	Odometry	0.0099	0.0196	0.0194
Rotation	EKF	0.0022	0.0125	0.0003

Table 8 gathers the mean, maximum and final position and orientation errors of the three tests. The clearest improvement was in the square, where the EKF reduced all six metrics. In the circular motion, the mean and final position errors and the three angular errors decreased, although the maximum position error was higher. During the rotation, the orientation improved, while the odometry kept lower position errors.

6.4 Closed-loop tracking with EKF feedback

In Section 6.3, the localizer was evaluated independently, without using the trajectory generator or controller. In this subsection, by contrast, the complete system is analyzed while tracking the square with 4 m sides, using the pose estimated by the EKF as feedback.

The configuration of experiment 4 from the previous sweep was kept, in order to evaluate the change of feedback without changing the controller constants at the same time

$$K_\rho = 0.5, \quad K_\alpha = 0.6, \quad lookahead = 0.5 \text{ m}.$$

The controller received the pose published on `/pose` instead of the one published on `/robot/odometry`. Since the EKF estimate is expressed in `map` and the desired trajectory in `odom`, the pose was transformed to `odom` through TF before computing the error. The selection of the reference through the *lookahead*, the error computation and the proportional law that generates the commands on `/robot/cmd_vel` were left unchanged.

To build the plots, it was checked that all signals belonged to the same run. In the recorded data, the EKF pose was also transformed to the `odom` frame. The ground truth times were taken as the reference to compare the signals instant by instant. When the odometry or the EKF pose had no sample at one of those instants, an intermediate position was computed by linear interpolation between the immediately preceding and following samples. This computation assumes that each position coordinate varies linearly between the two samples. Only the common time interval of each comparison was considered.

The three direct position metrics between the EKF and the ground truth in Table 9 were computed with the positions aligned in this way, in the window $[0, 69.70]$ s. The direct comparison between odometry and ground truth used the window $[0.05, 69.70]$ s, because the first recorded odometric pose corresponds to $t = 0.05$ s. The recorded duration of the commands was 69.65 s, from $t = 0.05$ s to the stop at $t = 69.70$ s.

6.4.1 Resulting trajectory

Figure 21 compares the desired trajectory, the ground truth, the pose estimated by the EKF and the odometry, all expressed in `odom`. In this run, the odometry is recorded passively: the feedback of the controller comes from the EKF.

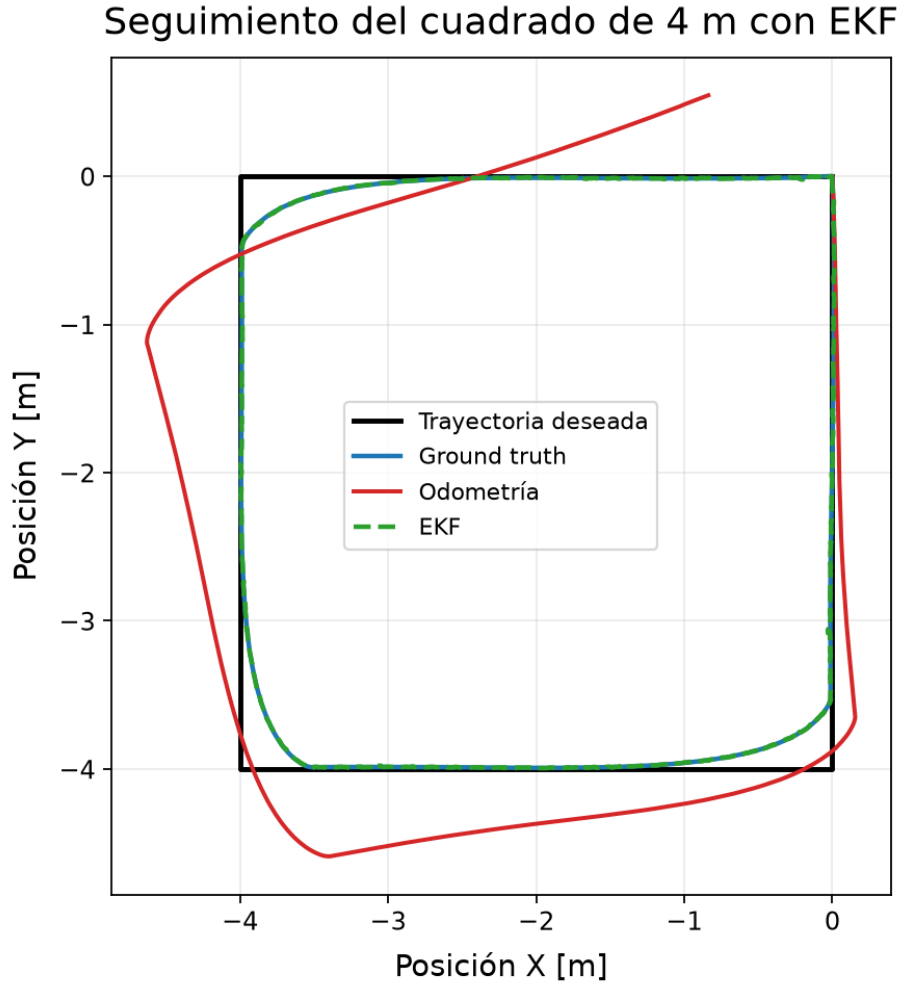


Figure 21: Trajectory tracking using EKF-based localization.

The trajectory estimated by the EKF stays practically on top of the ground truth for almost the entire path, while the odometry shows a visible separation on the left side, in the lower area and in the final segment. Two aspects should be separated in this comparison: the closeness between the EKF and the ground truth shows how well the localization works, while the distance between the ground truth and the desired trajectory shows how well the controller follows the path. Even though the filter represents the actual pose well, corner cutting still appears at the changes of direction.

6.4.2 Time evolution of the pose

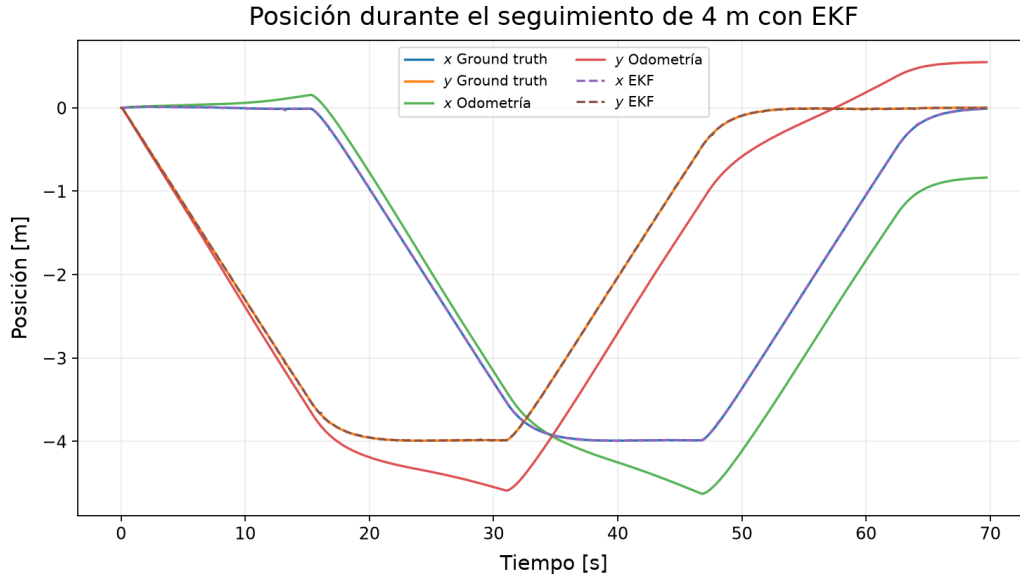


Figure 22: Time evolution of the position during tracking with the EKF.

During the first segment, the y component decreases to values close to -4.0 m, while x stays around zero (Figure 22). Then x decreases to a similar magnitude; next, y returns toward zero and, finally, so does x . The EKF curves closely follow the ground truth during these four stages. The passive odometry drifts away at the extremes and does not return to the origin: at the end, it keeps a negative offset in x and a positive one in y .

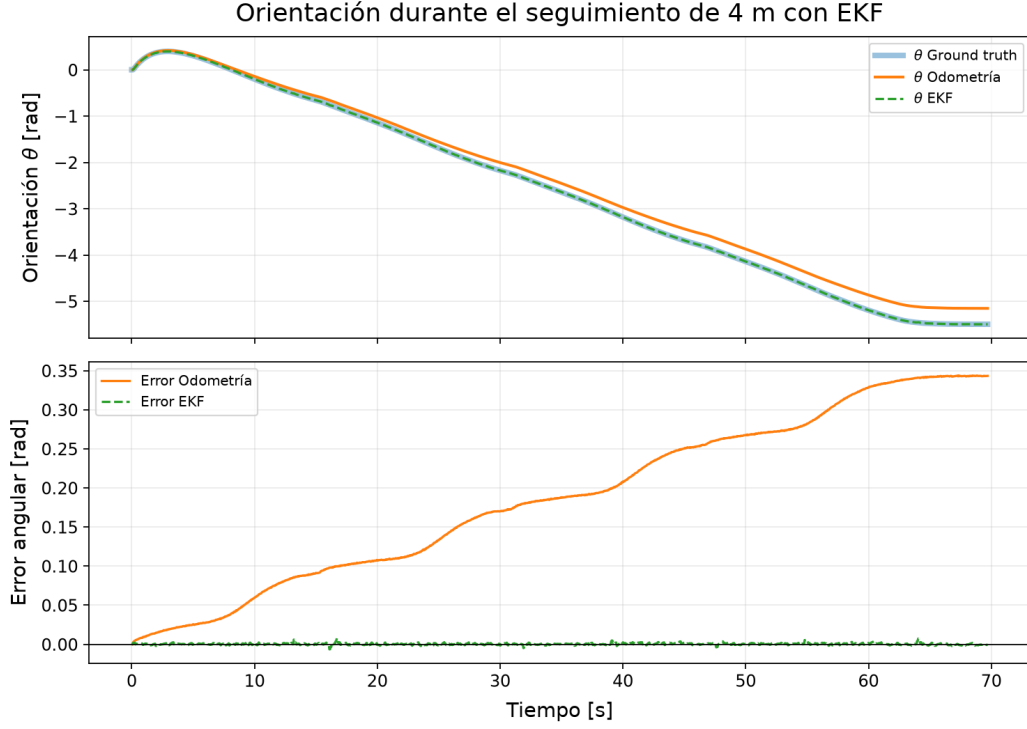


Figure 23: Time evolution of the orientation and normalized angular error with respect to the ground truth during tracking with the EKF.

The upper panel of Figure 23 shows the continuous evolution of the orientation, without introducing jumps between π and $-\pi$. The changes of slope coincide with the transitions between the different segments of the path. The EKF estimate follows the ground truth, while the odometry gradually drifts away.

The lower panel magnifies these differences with respect to the ground truth. For the odometry, the angular error grows in stages and keeps a bias at the end, while with the EKF it stays around zero, with small oscillations and a few isolated peaks. This error measures the difference between the estimated and the actual orientation; it is different from the angular error with respect to the radial reference used to compare the seven control configurations.

6.4.3 Path error

After comparing the pose estimates, the tracking was evaluated through the distance to the closest sampled point of the desired trajectory. The same geometric criterion as in Equation 17 was used, without matching the robot position to a reference point by time.

The computation was done twice: once taking (x_k, y_k) from the ground truth pose and once taking the pose estimated by the EKF. In both cases, the error was measured with respect to the complete trajectory and not to the active target of the controller.

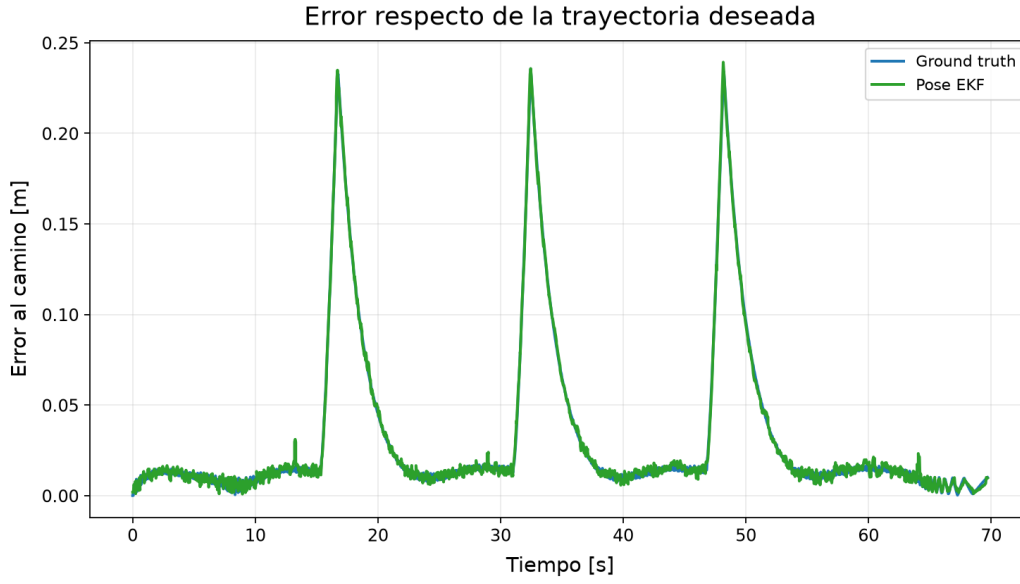


Figure 24: Error with respect to the desired trajectory during tracking with the EKF, computed with the ground truth and with the pose estimated by the filter.

In Figure 24, both curves stay almost on top of each other. Three main peaks can be seen, associated with the changes of direction, close to 0.24 m. Between them, the error returns to low values.

Table 9: Tracking metrics using EKF-based localization.

Metric	Value
Mean path error (ground truth)	0.034 m
Maximum path error (ground truth)	0.236 m
Final path error (ground truth)	0.010 m
Mean path error (EKF)	0.034 m
Maximum path error (EKF)	0.239 m
Final path error (EKF)	0.010 m
Recorded command duration	69.65 s
Mean position error, EKF vs. ground truth	0.003 m
Maximum position error, EKF vs. ground truth	0.021 m
Final position error, EKF vs. ground truth	0.000379 m

The position difference between the EKF and the ground truth is small compared with the path error at the corners (Table 9).

6.4.4 Velocities sent to the robot

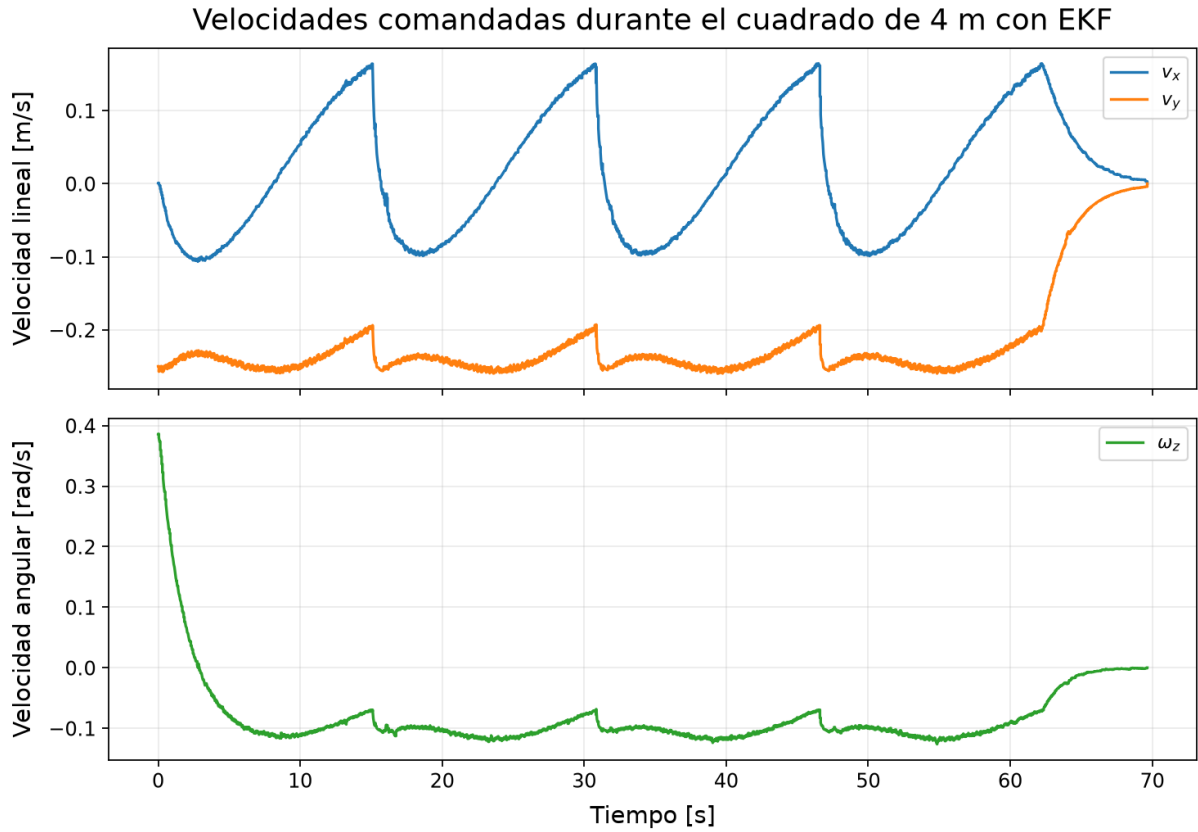


Figure 25: Velocities sent to the robot during tracking with the EKF.

The velocities sent to the robot remain bounded throughout the run (Figure 25). The v_x component changes sign in each segment, while v_y stays negative and tends to zero at the end. As for ω_z , after the initial transient the negative sign predominates and it also converges toward zero at the end. Smaller oscillations appear throughout the run, together with the successive corrections of the controller.

6.4.5 Evolution of the covariance

The variances of x , y and θ were recorded, and their standard deviations were computed by taking the square root. The position variances are expressed in m^2 and the angular one in rad^2 ; the corresponding standard deviations are expressed in meters and radians. Figure 26 shows only the latter.

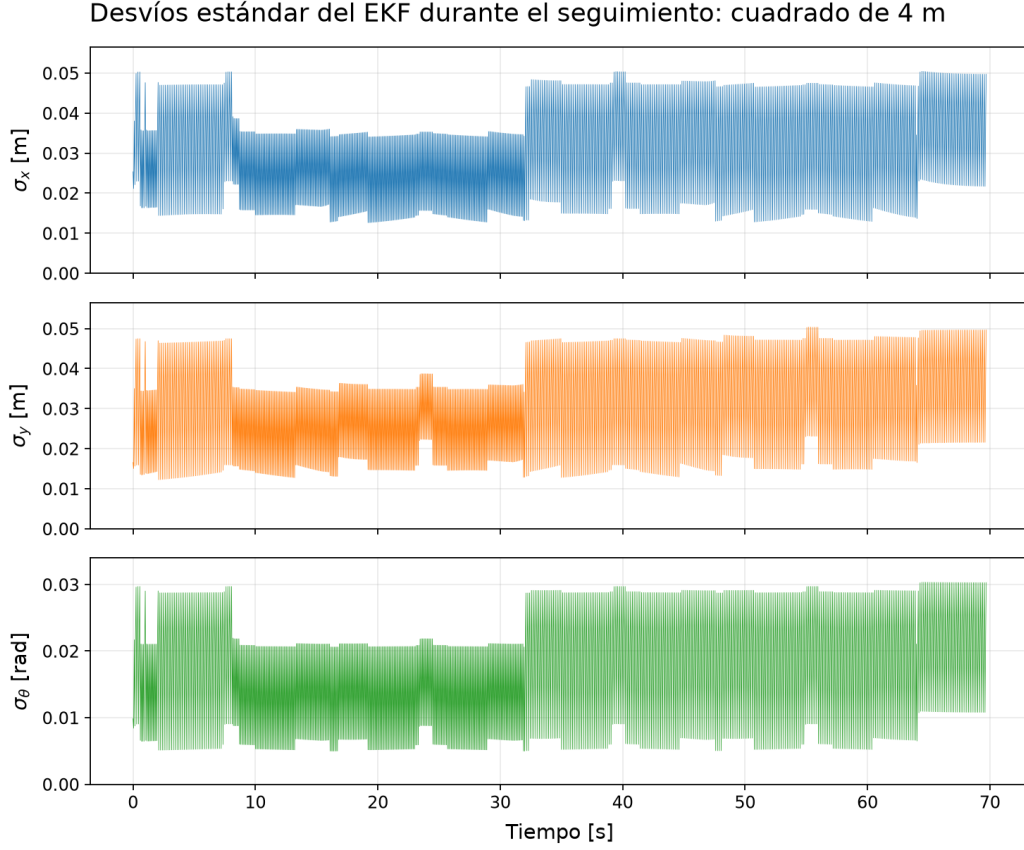


Figure 26: Standard deviations of the pose estimated by the EKF during the 4 m tracking run. The last published sample of each timestamp is kept, without smoothing.

Since predictions and corrections can be recorded with the same timestamp, the maxima in this figure are not equivalent to the maxima computed over all messages. In addition, the first published point already comes after corrections and does not represent the initial covariance Σ_0 .

Repeated oscillations between the prediction and correction stages can be seen, without sustained growth during the run. The maxima in the plot are approximately 0.0505 m for the x deviation, 0.0504 m for y and 0.0303 rad for the angle. The final values are 0.0497 m, 0.0497 m and 0.0303 rad for x , y and θ , respectively, close to the upper part of those oscillations.

During tracking, between three and five landmarks were associated per message, so the available information varied between updates. The covariance does not measure the distance between the robot and the desired trajectory, but the uncertainty of the EKF about the estimated pose. The ground truth can stay close to the square even if the covariance increases: lower confidence in the estimate does not necessarily mean a larger tracking error at that instant.

Figures 27, 28 and 29 illustrate the RViz2 visualization at the start, at an intermediate instant and at the end of an additional 4 m tracking run with the planar 2D detector, carried out to visualize the covariance. The purple region represents the uncertainty of the pose estimated by the EKF. The red posts correspond to the known map and the yellow points to the detected landmarks.

The white scan points are shown at the height of the laser, while the detected centers, in yellow, are projected onto the $z = 0$ plane of the `base_link` frame. Because of this height difference, in the oblique RViz2 view the yellow points may appear outside the white arcs that represent the visible surface of the posts.

To place the detected points in the `map` frame, RViz2 uses the following chain of transformations

$$\text{map} \rightarrow \text{odom} \rightarrow \text{base_link} \rightarrow \text{detected point}.$$

The `odom` $\rightarrow$ `base_link` transformation is obtained from the odometry and can accumulate error during the run, so the detected points may appear shifted with respect to the posts of the map. The EKF performs the correction using the distance and angle measurements published on `/landmarks` and the known positions from `/posts`; the *point cloud* is used only as a visual aid.

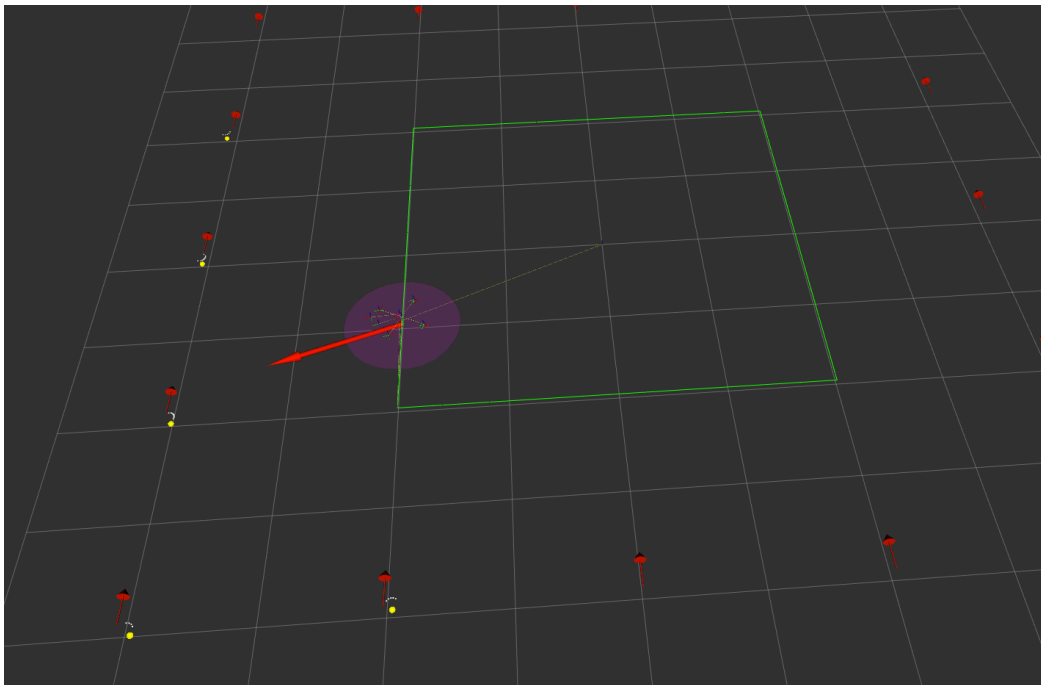


Figure 27: RViz2 visualization of the covariance of the estimated pose at the start of the tracking run.

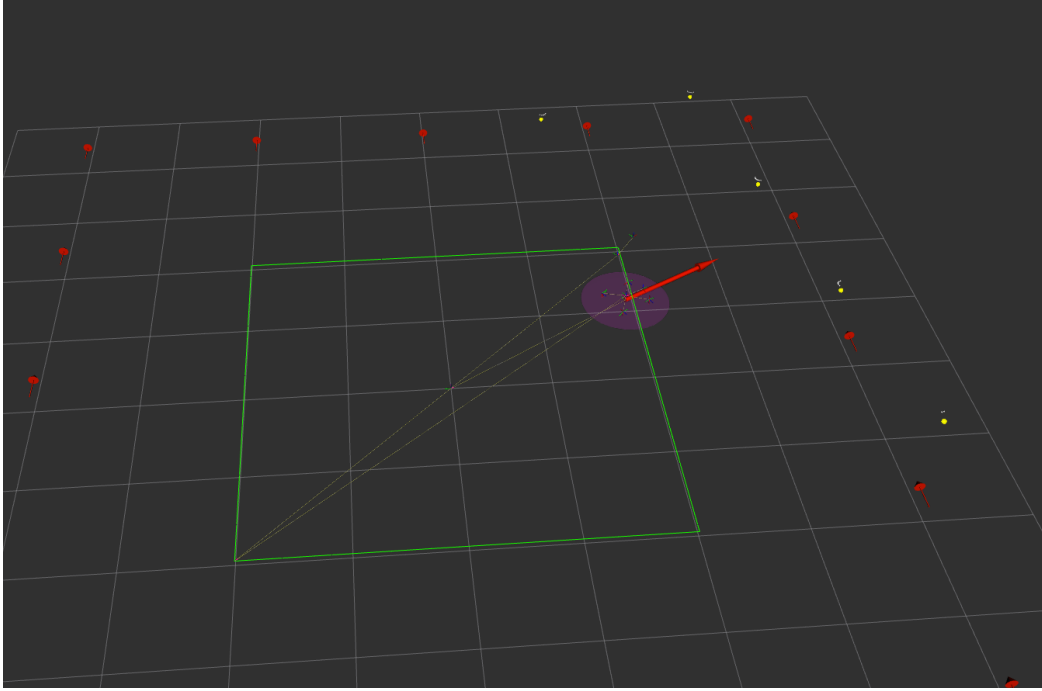


Figure 28: RViz2 visualization of the covariance of the estimated pose at an intermediate instant of the tracking run.

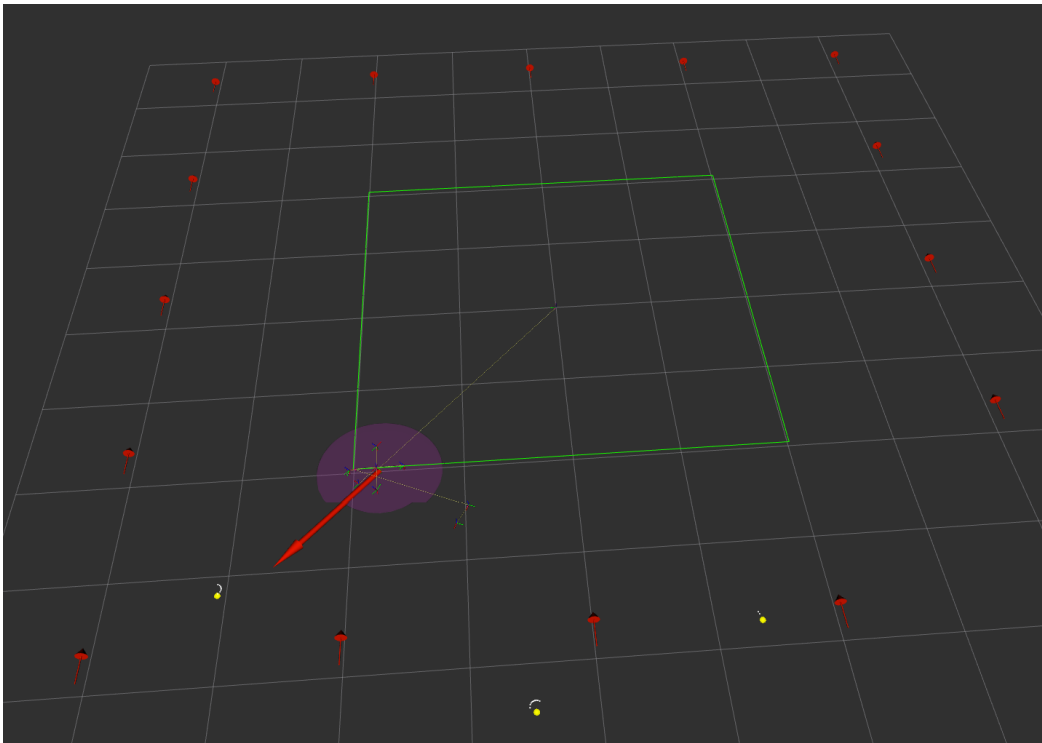


Figure 29: RViz2 visualization of the covariance of the estimated pose at the end of the tracking run.

The final covariance values were obtained from the `PoseWithCovarianceStamped` message. The EKF state consists of x , y and θ , so the state covariance is represented in the

equations by the 3×3 matrix Σ_t . The message, on the other hand, stores the pose covariance as an array of 36 elements that represents a 6×6 matrix row by row. To distinguish the two matrices, this matrix is denoted P in this section. Using zero-based indices, the variances of x , y and yaw correspond to P_{00} , P_{11} and P_{55} ; in the message array they occupy positions 0, 7 and 35, respectively.

Since the pose published on `/pose` is expressed in `map`, the variances of x and y correspond to the axes of that frame, while the yaw variance corresponds to the orientation with respect to the same frame. In the data used for the comparisons, the pose was transformed to `odom`; the covariance remained expressed in `map`.

In addition to the covariance, the number of landmark observations received, associated and discarded was recorded. Table 10 summarizes these counts and the variances of the last publication. Only the command window $[0.05, 69.70]$ s, lasting 69.65 s, was considered; the observations after the stop were not included.

Table 10: Association diagnostics and final variances during the 4 m tracking run with the EKF.

Metric	Value
Observations received	2371
Observations associated	2371
Observations discarded	0
Association percentage	100%
Final variance P_{00} [m ²]	0.002471
Final variance P_{11} [m ²]	0.002466
Final variance P_{55} [rad ²]	0.000916

The 100% figure indicates that the filter accepted an association for every observation received in that window. Observations were counted, not distinct posts. During that window, the maximum distance between a projected detection and the associated post was approximately 0.068 m, below the 1.0 m threshold. The fact that all of them were associated does not imply that their number per message was constant or that the measurements were free of noise, nor does it guarantee that each correspondence is correct.

With the rounding used, the square roots of the final variances match the final values shown in the standard deviation plot.

6.4.6 Comparison with odometry-based tracking

Tracking with the EKF was compared with the odometric run of experiment 4, presented in Table 6 and analyzed in Section 6.2.1. Both used the same control parameters.

To evaluate the actual tracking, the ground truth of each run is compared with the reference in the windows $[0, 67.95]$ s for the odometric run and $[0, 69.70]$ s for the EKF run.

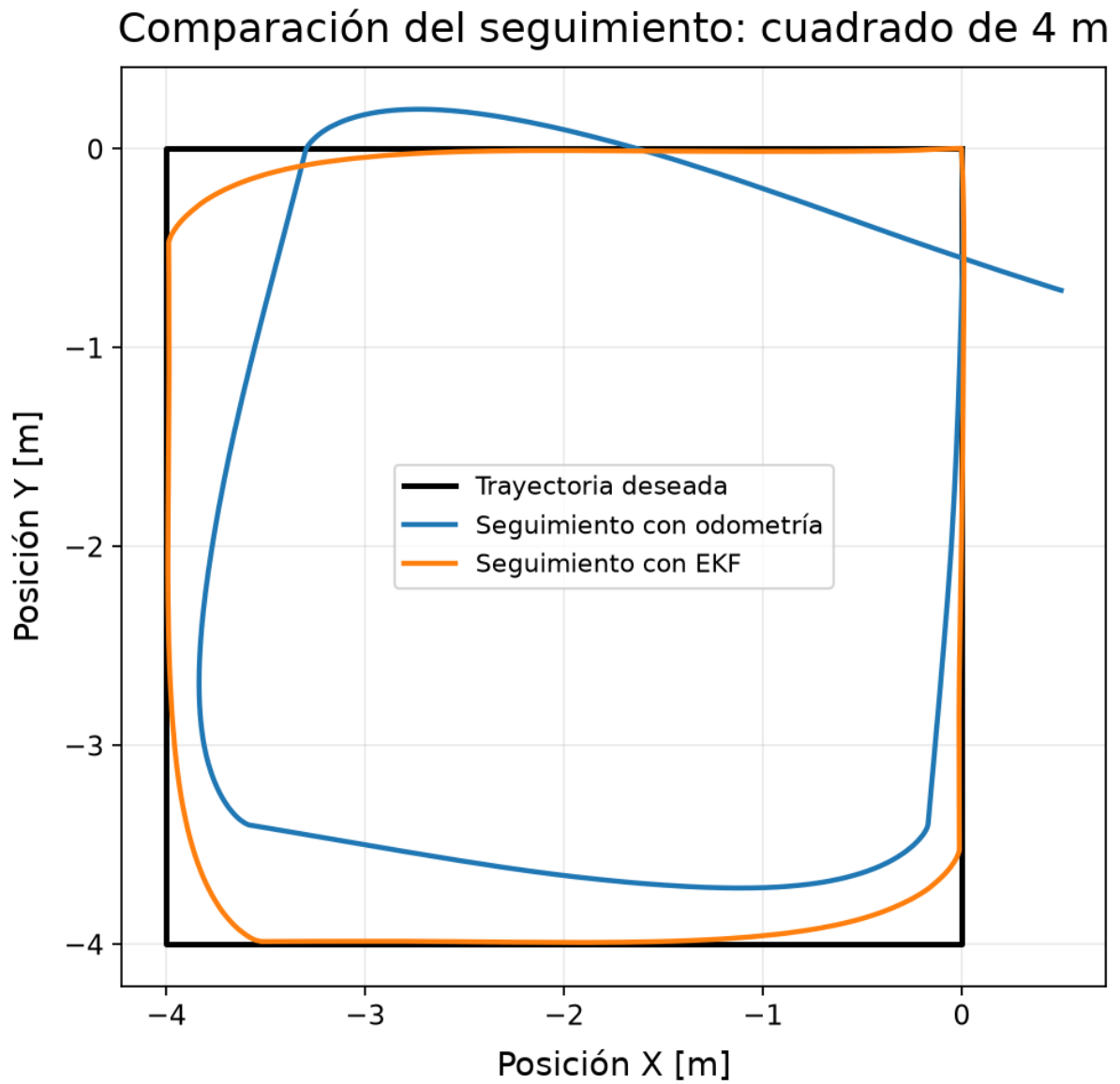


Figure 30: Comparison of trajectories between odometry-based tracking and EKF-based tracking.

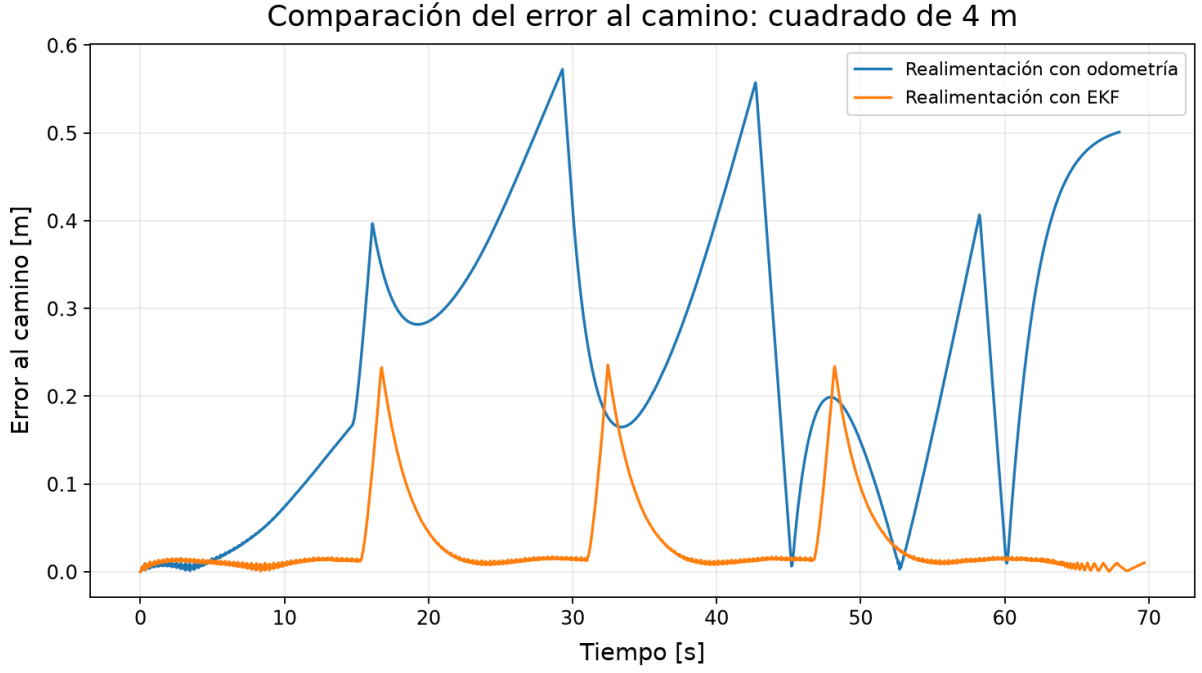


Figure 31: Comparison of the path error between odometry-based tracking and EKF-based tracking.

In Figure 30, both runs reproduce the general shape of the reference trajectory. However, tracking with the EKF stays closer to the reference on the left side, in the lower area and, above all, at the end.

With the EKF, the main errors are concentrated at three changes of direction and then return to low values, as shown in Figure 31. In the run with odometric feedback, the error computed with the ground truth also grows within the segments, and the robot ends far from the path. The curves cross at some moments, so the lower overall errors with the EKF do not represent a uniform improvement at every sample.

Table 11: Comparison of metrics between tracking with odometry and tracking with the EKF.

Metric	Odometry	EKF
Mean path error (GT) [m]	0.242	0.034
Maximum path error (GT) [m]	0.573	0.236
Final path error (GT) [m]	0.501	0.010
Recorded command duration [s]	67.90	69.65

Table 11 shows lower mean, maximum and final path errors when the EKF pose is used as feedback. The duration increased by 1.75s, a small difference compared with the total time. In these two runs, the change of feedback made it possible to follow the reference more closely, although errors persisted at the corners.

7 Developed system

The system was organized into ROS2 nodes to separate the kinematic model, the odometry, trajectory tracking and EKF-based localization. Figure 32 summarizes the nodes and the main topics used in the implementation.

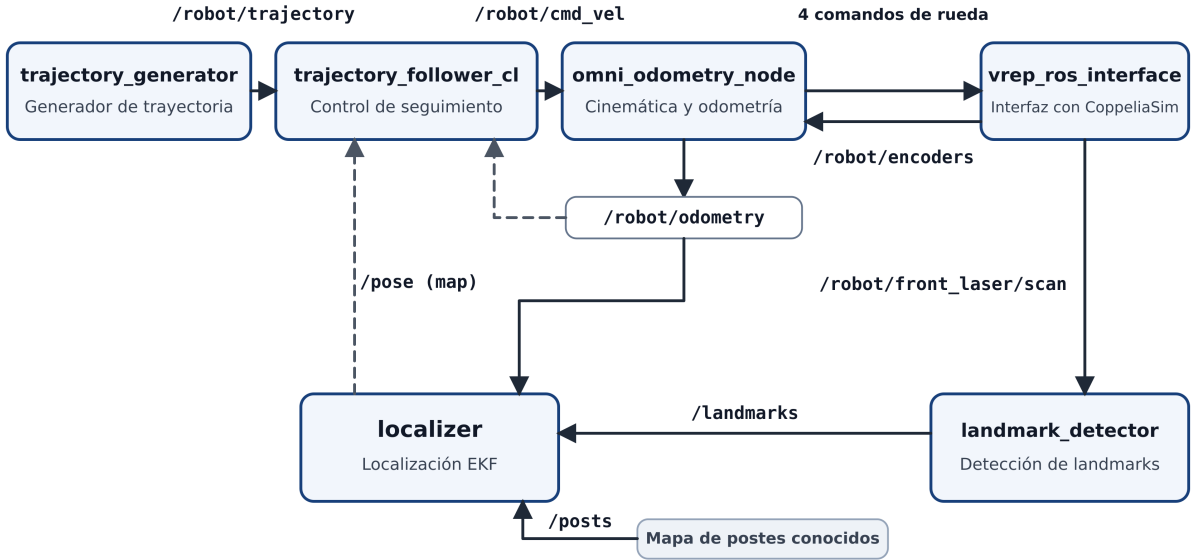


Figure 32: Nodes and main information flows of the system developed in ROS2. The dashed lines indicate the alternative feedback sources of the controller; only one is used in each run.

During the runs reported here, the nodes communicated as shown in the diagram. In the submitted version, the odometric controller obtains the pose by looking up the `map` → `base_link` transformation and transforms it to `odom` before computing the error, instead of receiving it through `/robot/odometry`. This change modifies how the odometric pose is obtained, not the model that computes it. The EKF feedback mode still reads `/pose`.

The kinematic node receives a `geometry_msgs/msg/Twist` command through the `/robot/cmd_vel` topic. From that message, it applies inverse kinematics, computes the target angular velocities of the four wheels and publishes them on their individual topics. In addition, it receives the measurements from `/robot/encoders`, applies forward kinematics and integrates the pose to publish the odometry and the transformation between `odom` and `base_link`.

The controller receives the desired trajectory through `/robot/trajectory` and selects an instantaneous reference pose, published on `/goal_pose`. It then compares that reference with the estimated pose and generates a new velocity command on `/robot/cmd_vel`. In the first tests, it used the odometry as feedback, while in the final stage it worked with the pose corrected by the EKF.

Detection and localization are carried out in two stages. First, the landmark detector receives the scans from `/robot/front_laser/scan` as `LaserScan` messages and transforms the points from the `front_laser` frame to `base_link` using the full TF. It then projects them onto the xy plane by setting $z = 0$. The resulting observations are published on

`/landmarks` as `robmovil_msgs/msg/LandmarkArray` messages, with planar distance and angle expressed with respect to the `base_link` frame.

Then, the EKF localizer receives those observations together with the velocities obtained from the odometry published on `/robot/odometry`, associates the detected landmarks with the map of known posts published on `/posts` and corrects the pose when it finds a valid correspondence.

The pose corrected by the EKF is published on `/pose`, in the `map` frame. In addition, the localizer publishes through TF the `map` $\rightarrow$ `base_link_ekf` transformation, which represents the pose estimate obtained by the filter. The controller and the analysis of results work in the `odom` frame, so the EKF pose is transformed from `map` to `odom` before it is used to control the robot or to compare the signals.

The `base_link` frame represents the chassis according to the odometry, while `base_link_ekf` represents the same chassis according to the corrected pose. In turn, the `base_link` $\rightarrow$ `front_laser` transformation, published by the scene, places the LiDAR sensor with respect to the chassis.

The main topics used were:

- `/robot/cmd_vel`: chassis velocity command, sent as `geometry_msgs/msg/Twist` messages;
- `/robot/encoders`: encoder measurements;
- `/robot/odometry`: estimated odometry, whose velocity components are used as input for the prediction of the EKF localizer;
- `/robot/front_laser/scan`: LiDAR scans received by the landmark detector;
- `/landmarks`: landmark observations published by the detector;
- `/landmarks_pointcloud`: estimated centers of the landmarks detected in each scan, used for visualization in RViz2;
- `/posts`: map of known posts;
- `/pose`: pose estimated by the localization system, published in the `map` frame;
- `/robot/trajectory`: desired trajectory received by the controller;
- `/ground_truth/target_path`: desired trajectory used for logging and visualization;
- `/robot/ground_truth`: actual pose of the robot provided by CoppeliaSim, used to compute the actual velocities in the validation of the kinematic model and to carry out the experimental comparisons;
- `/goal_pose`: instantaneous reference pose selected by the controller;
- `/robot/front_left_wheel/cmd_vel`, `/robot/front_right_wheel/cmd_vel`, `/robot/rear_left_wheel/cmd_vel` and `/robot/rear_right_wheel/cmd_vel`: target angular velocities of the wheels.

The logging node stored the signals used to compute the metrics and generate the plots.

The system was run with ROS2 Humble and CoppeliaSim 4.7.0 rev2, using the scene `codigo/simulacion/omni_ekf.ttt` included in the submission. The ROS2 nodes and RViz2 must be started before the simulation begins in order to receive the post map published at startup.

With the packages built and the environment sourced, EKF-based tracking is started with:

```
ros2 launch robmovil_ekf seguimiento_4m.launch.py \
  feedback_source:=EKF
```

To use odometric feedback, change it to `feedback_source:=ODOMETRY`.

8 Conclusions

The kinematic model responded reasonably for motions in v_x , v_y and ω_z , and forward kinematics made it possible to reconstruct the chassis velocity from the encoders. Limitations were also observed at high velocities and for simultaneous combinations in x and y : these results suggest that the response depends not only on the Mecanum geometry, but also on the capacity of the actuators of the simulated robot.

The odometry was adequate as a first estimate and made it possible to close the control loop. However, its difference from the ground truth grew as the run progressed, mainly in position, because small errors in the encoder measurements and in the reconstruction of the motion accumulate during integration.

No configuration was best on all metrics. Among the configurations tested, low values of K_ρ and of the *lookahead* reduced the mean and maximum errors, but increased the travel time. High values reduced the time, although they produced larger errors and cut the trajectory more at the changes of direction. The configuration with the lowest mean and maximum errors did not have the lowest final error. The variations of K_α also affected the position: with $K_\alpha = 0.6$, the path errors and the mean angular error were reduced compared with experiment 1, with practically the same duration.

The three manual tests made it possible to evaluate localization under longitudinal, lateral and rotational motions. In the circular trajectory, the EKF reduced the mean and final position errors, although it showed isolated peaks and a higher maximum error. The clearest improvement was obtained in the manual square: the EKF reduced the mean, maximum and final errors in both position and orientation, and stayed close to the ground truth during the four segments. In the in-place rotation, the EKF reduced the angular error, but the odometry kept a lower position error.

With the EKF pose as feedback, the path error computed with the ground truth was lower than in the odometric run of experiment 4, on the same 4 m square and with the same parameters. The mean error went from 0.242 m to 0.034 m, the maximum from 0.573 m to 0.236 m and the final from 0.501 m to 0.010 m. The times were similar: 67.90 s for the odometric run and 69.65 s for the EKF run, a difference of 1.75 s. In relative terms, the

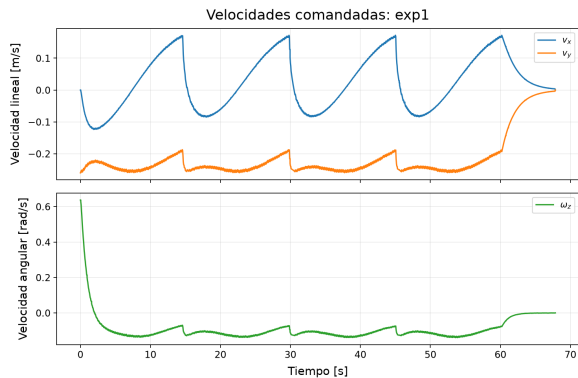
mean error decreased more than the maximum: between the changes of direction, the tracking stayed close to the reference, although deviations persisted at the corners.

As future work, it would be useful to repeat the tuning of K_ρ , K_α and the *lookahead* with the EKF in the loop, since the current parameters were selected through runs with odometric feedback. A variable *lookahead* could also be tested, and the velocity constraints observed in the robot could be included explicitly, which could reduce the errors that still appear at the changes of direction while keeping the travel time at similar values.

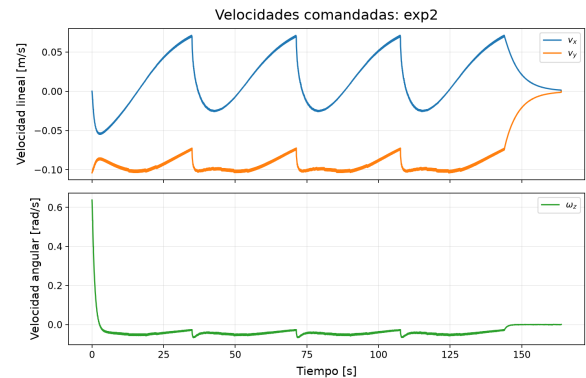
Finally, the work made it possible to integrate the kinematic model, the odometry, closed-loop control and EKF-based localization into a single system. The results show that the platform can follow trajectories in simulation and that the use of landmarks generally improves the pose estimate, although its effect varies depending on the motion and the component analyzed.

A Velocities of the control experiments

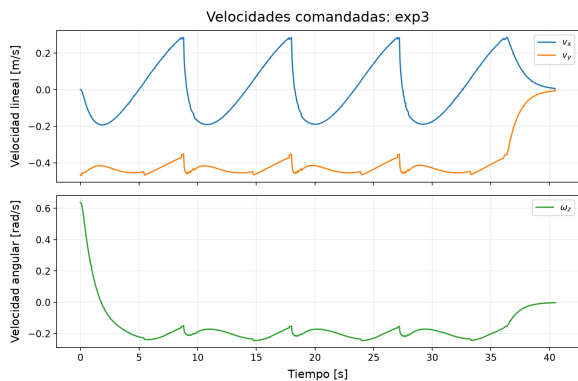
This appendix presents the linear and angular velocities sent to the robot during the seven configurations evaluated in the controller selection.



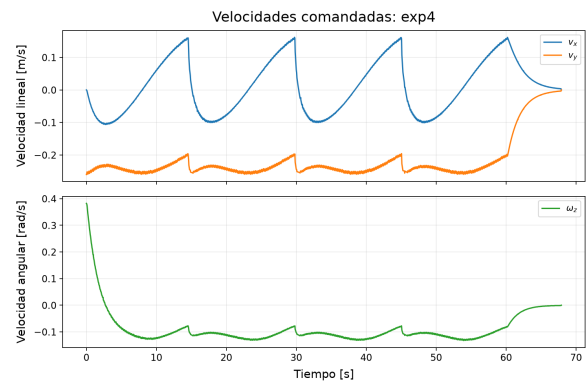
(a) exp1: $K_\rho = 0.5$, $K_\alpha = 1.0$, *lookahead* = 0.5.



(b) exp2: $K_\rho = 0.2$, $K_\alpha = 1.0$, *lookahead* = 0.5.

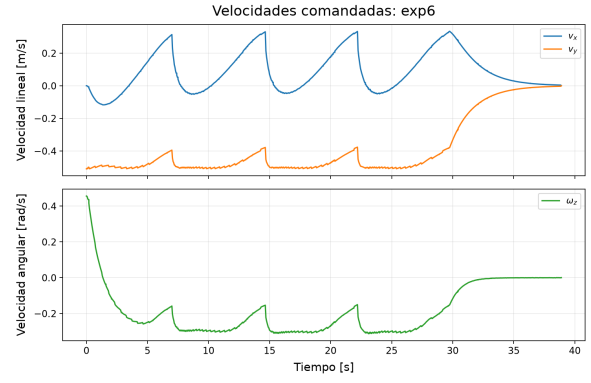
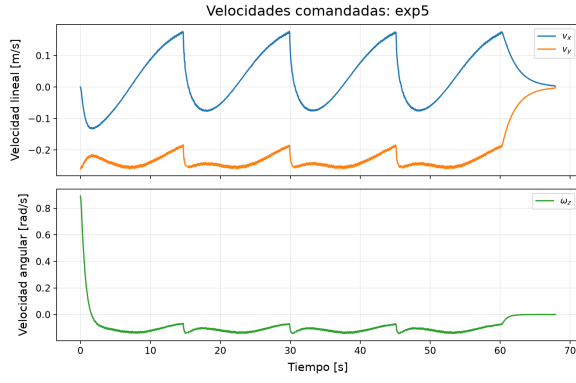


(c) exp3: $K_\rho = 0.9$, $K_\alpha = 1.0$, *lookahead* = 0.5.

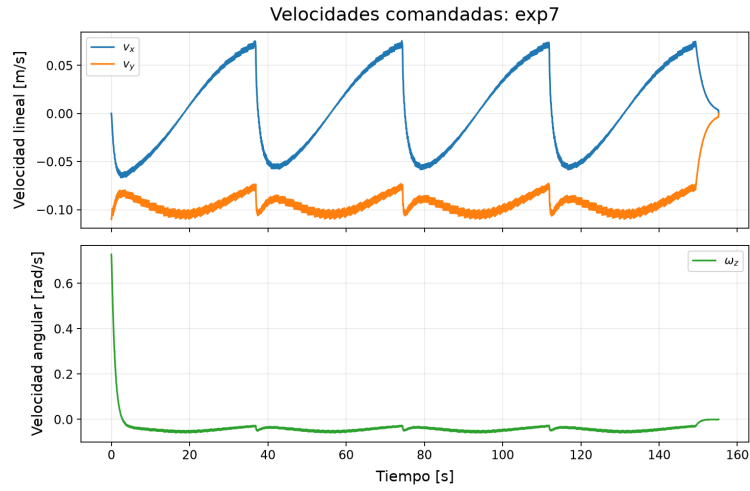


(d) exp4: $K_\rho = 0.5$, $K_\alpha = 0.6$, *lookahead* = 0.5.

Figure 33: Velocities sent to the robot in experiments 1 to 4.



(a) exp5: $K_\rho = 0.5$, $K_\alpha = 1.4$, *lookahead* = 0.5. (b) exp6: $K_\rho = 0.5$, $K_\alpha = 1.0$, *lookahead* = 1.0.



(c) exp7: $K_\rho = 0.5$, $K_\alpha = 1.0$, *lookahead* = 0.2.

Figure 34: Velocities sent to the robot in experiments 5 to 7.

B Actual pose and control targets

Figures 35, 36 and 37 compare the components of the actual pose with those of the active target during the seven control configurations. The actual pose was obtained from `/robot/ground_truth` and the target from `/goal_pose`, where the controller publishes the pose selected through the lookahead. Both are expressed in `odom`.

The original timestamps of each signal and the last publication of each repeated timestamp were kept. Each comparison was limited to the common interval of both logs, which ends at the last target sample, before the stop. The curves were not interpolated against each other: the target was represented as steps and the ground truth through its recorded samples. To show the orientation without artificial jumps, the 2π wrap-arounds were removed. The vertical scales were kept the same for a given component across all runs; the time scales correspond to the duration of each log.

The target moves ahead of the robot according to the lookahead. For this reason, this comparison is not equivalent to the geometric distance to the path. Nor does it represent the internal error of the controller, which is computed using the odometric pose. At the end, the target reaches the closing point, but the ground truth curves do not end at that same position.

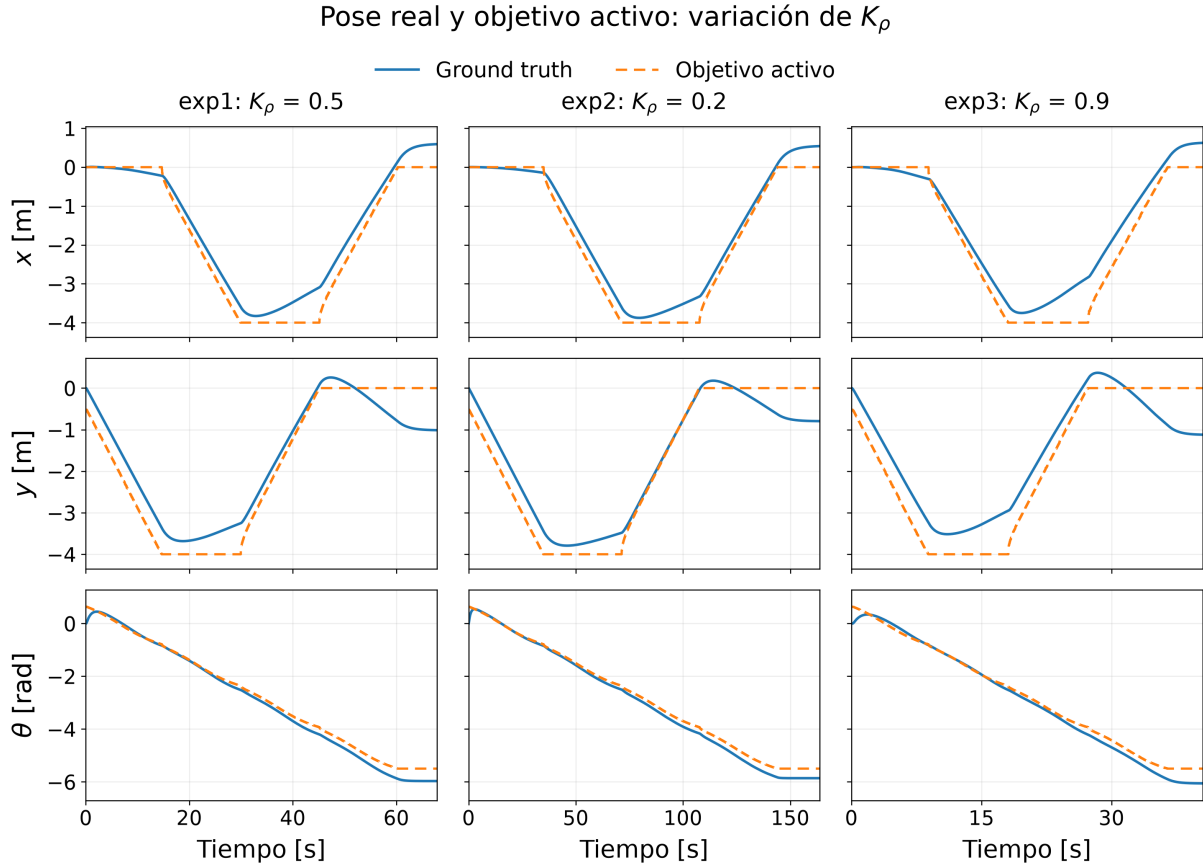


Figure 35: Actual pose and active target when varying K_ρ , with $K_\alpha = 1.0$ and a lookahead of 0.5 m. Each column corresponds to one run.

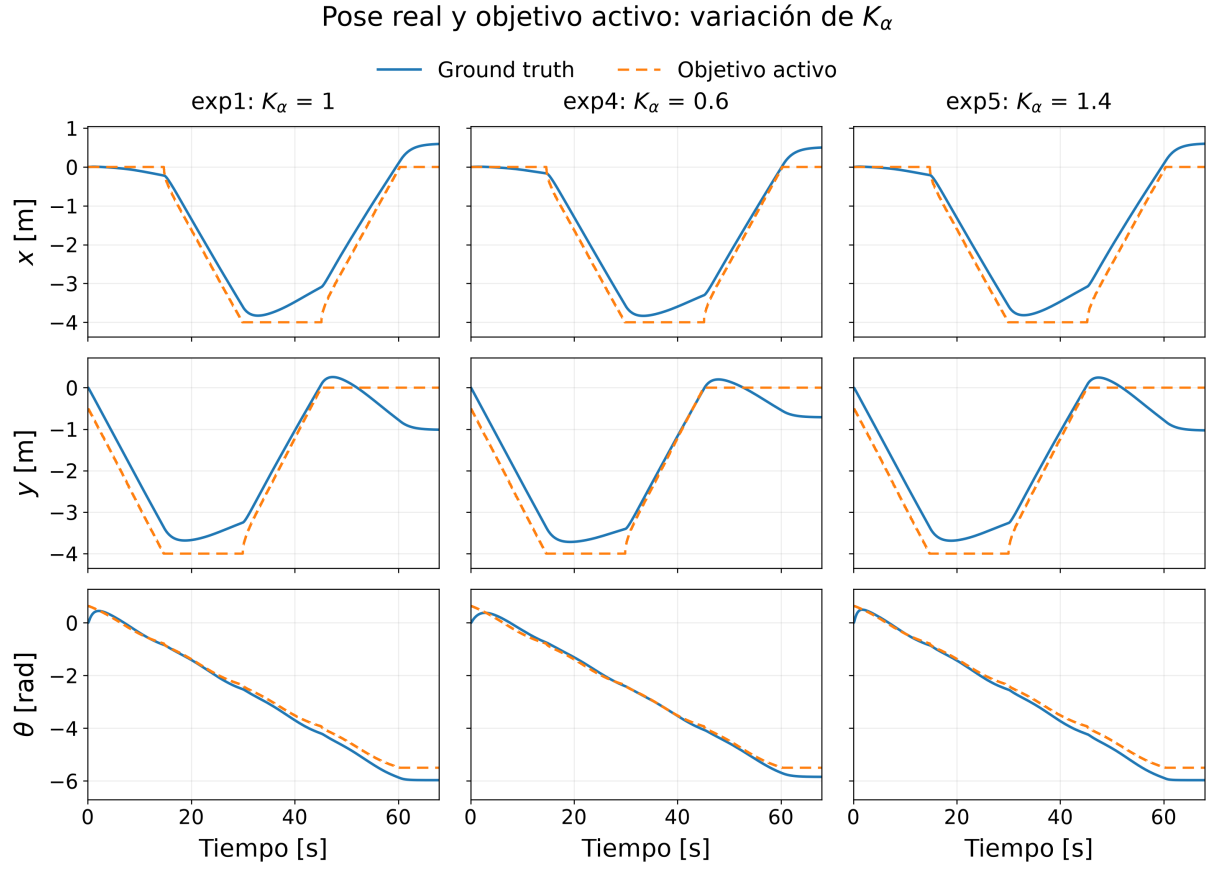


Figure 36: Actual pose and active target when varying K_α , with $K_\rho = 0.5$ and a lookahead of 0.5 m.

Pose real y objetivo activo: variación de lookahead

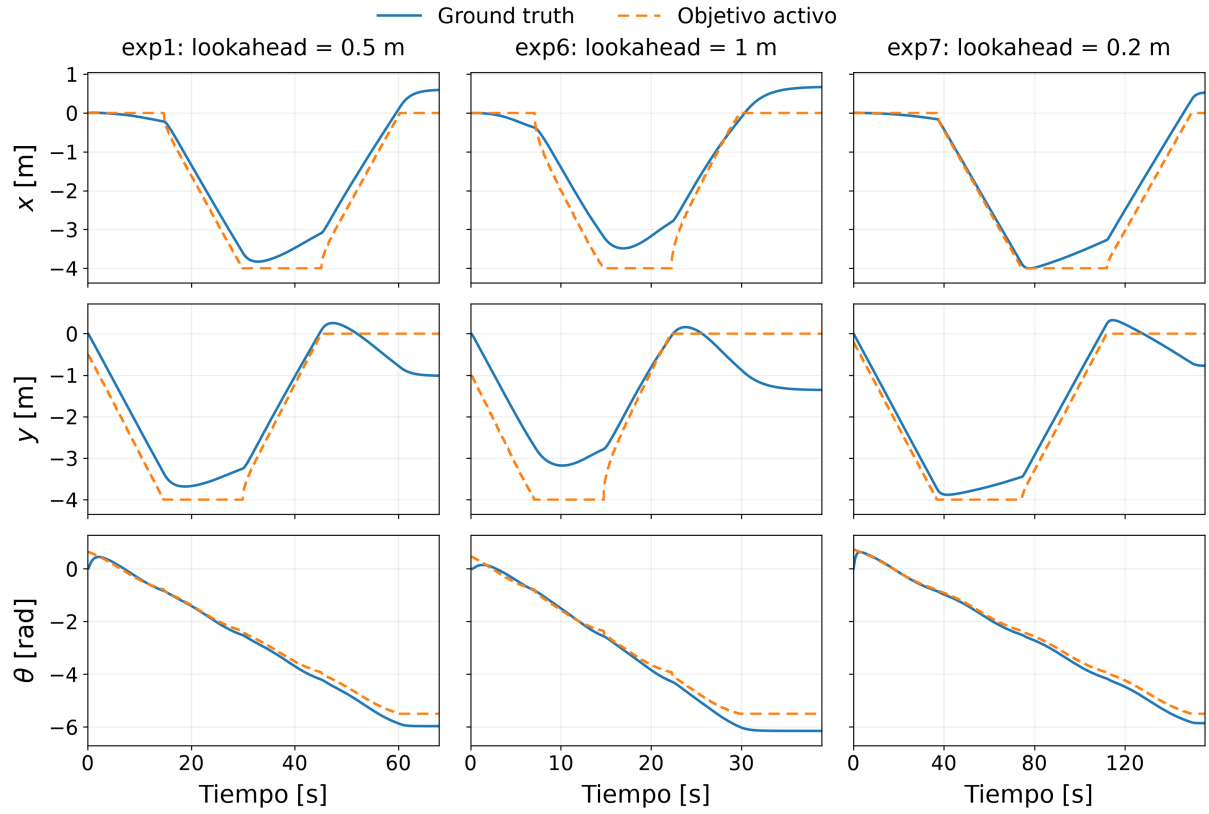


Figure 37: Actual pose and active target when varying the lookahead, with $K_\rho = 0.5$ and $K_\alpha = 1.0$.

References

- [1] H. Taheri, B. Qiao and N. Ghaeminezhad, “Kinematic Model of a Four Mecanum Wheeled Mobile Robot,” *International Journal of Computer Applications*, vol. 113, no. 3, pp. 6–9, 2015.